

STARTANLEITUNG ÖFFNEN →

Interaktiver PDF-Sprung. Startet nicht das C++-Spiel.

© 2026 Julian Krauß | JK-CD-DE-2026-003 | Entwurf 1.0

INHALTSVERZEICHNIS

NAVIGATION / SCHNELLZUGRIFF

Interaktives Inhaltsverzeichnis

Klicke auf eine Karte, ein Kapitel oder eine Seitenzahl. Alle Ziele sind direkt verlinkt.

ÜBERSICHT | C++-MODULE | DOKUMENTATION | PAP P01–P12

TEIL A · SPIELEN — S. 6
Gaming-UI & Bedienung
Startbildschirm, Steuerung, Spielregeln und Hilfe.

TEIL B · LERNEN — S. 11
C++ Schritt für Schritt
23 verständliche Programmiermodule, Code und Tests.

TEIL C · PLANEN — S. 35
Technische Dokumentation
Architektur, Tests, Persistenz, Entwurfsentscheidungen.

TEIL D · ERKLÄREN — S. 73
12 Programmablaufpläne
Alle Ablaufdiagramme einzeln in großem A3-Format.

DIREKT ZU DEN WICHTIGSTEN THEMEN

Spiel starten | Klassen & OOP | Gewinnprüfung | Tests | GitHub | Live-Demo

Hinweis: Gaming-Oberfläche = Konzept. Die C++-Lernversion ist ein Konsolenprojekt.

LERNPFAD / 23 MODULE

C++-Programmier-Anleitung

Vom ersten Compiler-Aufruf bis zu Dateiverarbeitung, Tests und GitHub.

ÜBERSICHT | C++-MODULE | DOKUMENTATION | PAP P01–P12

GRUNDLAGEN & SPIELLOGIK

00	Dein Weg zum fertigen C++-Spiel	12
01	C++-Werkzeuge auf Windows vorbereiten	13
02	Projektordner und Dateien anlegen	14
03	Erstes C++-Programm kompilieren	15
04	Klasse Player verstehen	16
05	Spielfeld mit 2 bis 4 Spielern aufbauen	17
06	Kurzhanteln in eine Spalte fallen lassen	18
07	Kleine Übung mit dem Board testen	19
08	Gewinnprüfung: vier in einer Reihe	20
09	Hauptmenü und Spielschleife programmieren	21
10	Spieleranzahl, Namen und Zugwechsel	22
11	Sonderbefehle in der laufenden Partie	23

PERSISTENZ, TESTS & ABSCHLUSS

12	Fallanimation und Gaming-Darstellung	24
13	Spielstand speichern und laden	25
14	Highscore dauerhaft speichern	26
15	Das ganze Projekt starten	27
16	Automatische und manuelle Tests	28
17	Häufige Fehlermeldungen lösen	29
18	GitHub und Projektdokumentation	30
19	Vom Konsolenprogramm zur Gaming-UI	31
20	Welche Funktionen gehören zu welchem PAP?	32
21	Typische Lehrerfragen, direkt beantwortet	33
22	Abgabe- und Lerncheckliste	34

Tipp: Mit Modul 00 anfangen und erst danach zur Spielsteuerung wechseln.

© 2026 Julian Krauß | JK-CD-DE-2026-003 | INTERAKTIVER INDEX · REV 1.0

NACHWEIS / 25 KAPITEL

Technische Dokumentation

Anforderungen, Architektur, Testplanung und Antworten auf Fachfragen.

ÜBERSICHT | C++-MODULE | DOKUMENTATION | PAP P01–P12

KONZEPT & KERNFUNKTIONEN

01	Zielbild & Anforderungen	36
02	So liest du den PAP	37
03	Objektorientiertes Konzept	46
04	Zustände, Regeln & Übergänge	47
05	Konkrete Designentscheidungen	48
06	Speicherformat & Validierung	49
07	Highscore & Fehlerbehandlung	50
08	Tests & Abnahmekriterien	51
09	Zusätzliche Prüffälle	52
10	Projektfahrplan 2026–2027	53
11	Anforderung → PAP → Test	54
12	Quellen, KI-Nutzung & Übergabe	55
13	Edition 2.0 | Feature-Upgrade	56

ERWEITERUNGEN & QUALITÄT

14	Pflicht und Zusatz trennen	57
15	Objektorientierte Architektur	62
16	Interface | Terminal-Konzept	63
17	Wirkung von Sonderbefehlen	64
18	Exakte Gewinnprüfung	65
19	Extras beweisbar testen	66
20	Speichern, Laden, Undo verbinden	67
21	Live-Demo mit Beweisführung	68
22	Antworten auf Lehrerfragen	69
23	Methoden & Pseudocode	70
24	Umsetzung nach Priorität	71
25	Design, Quellen & Status	72

P01 bis P12 auch innerhalb der Dokumentation erläutert; die großen A3-Pläne stehen separat am Ende.

© 2026 Julian Krauß | JK-CD-DE-2026-003 | INTERAKTIVER INDEX · REV 1.0

DIAGRAMME / DIREKTSPRÜNGE

12 Programmablaufpläne

Pflichtabläufe P01–P08 und optionale Komfortfunktionen P09–P12.

ÜBERSICHT | C++-MODULE | DOKUMENTATION | PAP P01–P12

KERNANFORDERUNGEN · P01–P08

P01	Hauptmenü	73
P02	Spielsteuerung	74
P03	Spielzug und Validierung	75
P04	Spielergebnis / Remis	76
P05	Gewinnprüfung	77
P06	Speichern	78

KERN / ERWEITERUNG · P07–P12

P07	Laden und Dateiprüfung	79
P08	Highscore	80
P09	Befehlssteuerung	81
P10	Zugvorschau	82
P11	Letzten Zug zurücknehmen	83
P12	Match Insights	84

Jeder Eintrag öffnet den einzelnen PAP in A3. Vergrößern im PDF bleibt verlustfrei.

© 2026 Julian Krauß | JK-CD-DE-2026-003 | INTERAKTIVER INDEX · REV 1.0

01 / SCHNELLSTART

So läuft eine Partie ab. Die Beispieloberfläche rechts ist ein grafisches Gestaltungskonzept.

01 **Spiel beginnen**
Im Hauptmenü „Neues Spiel starten“ wählen.

02 **Mitspieler festlegen**
2, 3 oder 4 Spieler auswählen und Namen eingeben.

03 **Spalte auswählen**
Gültige Spaltennummer eingeben. Die Hantel fällt nach unten.

04 **Gewinnen**
Vier eigene Symbole waagerecht, senkrecht oder diagonal verbinden.



ZIEL Vier Kurzhanteln in einer Linie. Die Regel bleibt **Vier gewinnt**.

WICHTIG / STATUS

Dieses PDF zeigt die **geplante** Oberfläche. Eine startbare C++-Anwendung liegt in diesem Paket nicht vor.

© 2026 Julian Krauß | JK-CD-DE-2026-003 | UI-/PAP-ENTWURF

02 / ANLEITUNG IN DER SPIELOBERFLÄCHE

Geplante Hilfeseite und Bedienkonzept



DESIGN-MOCKUP Die Schaltflächen und die Fallanimation sind in dieser PDF nur dargestellt.

© 2026 Julian Krauß | JK-CD-DE-2026-003 | UI-/PAP-ENTWURF

03 / STEUERUNG & SPIELREGELN

Pflichtfunktionen und geplante Komfortbefehle klar voneinander getrennt.

EINGABE	BEDEUTUNG	STATUS
1 ... N	Spalte setzen (je nach Spielfeld)	**Pflicht**
V N	Fallposition vorher anzeigen	**Optional**
U	Letzten gültigen Zug rückgängig	**Optional**
S	Aktuellen Spielstand speichern	**Optional**
I	Statistiken zur Partie zeigen	**Optional**
H	Kurzhilfe während der Partie	**Optional**

SPIELFELDGRÖSSEN

2 Spieler	**7 × 6**
3 Spieler	**9 × 8**
4 Spieler	**11 × 10**

SPIELENDE

Sieg: vier eigene Symbole in einer Reihe.
Remis: Spielfeld voll, kein Sieg.
Highscore: Sieg zählt **einmal**.

Darstellung: Kurzhanteln ersetzen die runden Steine nur optisch. Im Programm müssen die Spieler weiterhin eindeutig unterscheidbare Symbole und eine identische Gewinnregel haben.

© 2026 Julian Krauß | JK-CD-DE-2026-003 | UI-/PAP-ENTWURF

04 / TECHNISCH STARTEN

Die Anwendung ist ein C++-Projekt. Die folgende Startanweisung gilt nach Fertigstellung des Quellcodes.

01 / PROJEKT ERSTELLEN

Quellcode und Header sauber trennen, z. B. in **src/** und **include/**. Der Bauvorgang benötigt einen eingerichteten C++17-Compiler.

Beispiel Windows (PowerShell)

```
g++ -std=c++17 -linclude src/*.cpp -o vier-gewinnt.exe
```

02 / PROGRAMM STARTEN

Nach erfolgreichem Kompilieren wird die erzeugte Anwendung direkt über das Terminal geöffnet.

```
.\vier-gewinnt.exe
```

Dateinamen / Build anpassen.

GRAFIK-KONZEPT ≠ KONSOLENPROGRAMM

Das Aufgabenblatt verlangt zunächst ein **spielbares C++-Konsolenprogramm**. Die gezeigte 3D-Gaming-UI ist eine zusätzliche Designidee. Für eine echte Fensteroberfläche, grafische Kurzhanteln und flüssige Fallanimation wäre z. B. ein separates Rendering-Modul mit einer Grafikbibliothek erforderlich. Das ist **noch nicht implementiert**.

GUTER UMSETZUNGSWEG

Reihenfolge: Klassen Player / Board / Game → Zugvalidierung und Gewinnprüfung → Speichern/Laden und Highscore → optionale Befehle → optionales UI-Rendering. Die Spielregeln bleiben unabhängig von der Darstellung. Dadurch lässt sich die klassische Konsolenversion weiterhin prüfen.

© 2026 Julian Krauß | JK-CD-DE-2026-003 | UI-/PAP-ENTWURF

05 / PROGRAMMABLAUFPLÄNE IM ANHANG

P01–P08 bilden die Kernanforderungen ab. P09–P12 ergänzen optionale Komfortfunktionen.

PAP	BESCHREIBUNG	UMSETZUNGSART
P01	Hauptmenü	**Pflicht**
P02	Spielsteuerung	**Pflicht**
P03	Spielzug und Validierung	**Pflicht**
P04	Spielergebnis / Remis	**Pflicht**
P05	Gewinnprüfung	**Pflicht**
P06	Speichern	**Pflicht**
P07	Laden und Dateiprüfung	**Pflicht**
P08	Highscore	**Pflicht**
P09	Befehlssteuerung	**Optional**
P10	Zugvorschau	**Optional**
P11	Zug zurücknehmen	**Optional**
P12	Match Insights	**Optional**

LEHRER-FRAGEN

Warum wächst das Feld?
Für jeden Spieler über zwei: +2 Reihen und +2 Spalten.

Wie wird ein Sieg erkannt?
Vier Richtungen um den zuletzt gesetzten Stein prüfen.

DOKUMENTATION

Alle zwölf PAPs sind im A3-Anhang verlinkt. Die Gesamtfassung enthält zusätzlich die technische Dokumentation.

ALLE PAPs ÖFFNEN →

© 2026 Julian Krauß | JK-CD-DE-2026-003 | UI-/PAP-ENTWURF

INHALT

BERND BLINDOW GRUPPE
SCHULE AUSBILDUNG STUDIUM

PROJEKTAUFGABE 51 / SCHRITT FÜR SCHRITT

C++ PROGRAMMIEREN

VIER GEWINNT

GYM EDITION · Von der ersten Zeile bis zum Start



23 LERNMODULE / QUELLCODE + TESTS + PAP-ZUORDNUNG

Das Konsolenspiel ist ausführbar. Die 3D-Spieloberfläche bleibt ein Designentwurf.

Beinhaltet konkrete Terminalbefehle, Klassen und geprüfte Beispieltests.

JULIAN KRAUSS · ITA 2026/27 · BERND-BLINDOW-SCHULEN RAVENSBURG

DESIGN-ID: JK-CD-DE-2026-003 | © 2026 Julian Krauß

00 / PROGRAMMIER-ANLEITUNG

Dein Weg zum fertigen C++-Spiel

Erst die Spiellogik verstehen. Danach Extras ergänzen. Die Bilder zeigen das spätere Gym-Design, nicht bereits eine fertige 3D-Oberfläche.

Was du mit diesem Paket bekommst

Baustein	Inhalt
Diese PDF	Schritte, Erklärungen, Compilerbefehle, Tests und Hilfen
C++-Projekt ZIP	Komplette nachvollziehbare Konsolen-Referenz mit .h und .cpp
Übungen	Kleines Hallo-Welt-Programm und Test der Spielfeldklasse
PAP-Anhang	12 Programmablaufpläne im ergänzten Gesamtdokument

Arbeitsreihenfolge

- Schritt 01–03: Entwicklungsumgebung prüfen, Projekt öffnen, ersten Code ausführen.
- Schritt 04–08: Player, Board, Spielsteine und Gewinnprüfung verstehen.
- Schritt 09–14: Menü, Spielerzahl, Validierung, Spezialbefehle, Datei und Highscore.
- Schritt 15–20: Testen, Fehler beheben, dokumentieren und Gym-UI separat planen.

Wichtig für die Schule: Eine nachvollziehbare Lösung ist besser als eine große Codebasis, deren Funktionen du nicht erklären kannst. Erstelle nach jedem Abschnitt selbst einen kleinen Git-Commit.

01 / PROGRAMMIER-ANLEITUNG

C++-Werkzeuge auf Windows vorbereiten

Visual Studio Code ist dein Editor. Für das Übersetzen brauchst du zusätzlich einen C++-Compiler.

Schritt für Schritt

- Visual Studio Code öffnen und die Erweiterung „C/C++“ installieren.
- Prüfen, ob ein GCC-/MinGW-w64-Compiler mit C++17 verfügbar ist.
- In VS Code das Menü „Terminal > Neues Terminal“ öffnen.
- Im Terminal g++ --version eingeben und die Versionsausgabe kontrollieren.

```
g++ --version
```

Was du sehen solltest

Eine Compiler-Versionsnummer, beispielsweise „g++ (GCC) ...“. Steht dort „g++ wurde nicht gefunden“, ist der Compiler nicht installiert oder sein bin-Ordner fehlt im PATH.

> Windows-Hinweis: Verwende am besten Windows Terminal bzw. das integrierte VS-Code-Terminal. Nur mit Editor und Erweiterung allein kannst du noch keinen C++-Code übersetzen.

Deine Kontrollfrage

Was ist der Unterschied zwischen Editor (Code schreiben), Compiler (Quelltext übersetzen) und Terminal (Befehle starten)?

02 / PROGRAMMIER-ANLEITUNG

Projektordner und Dateien anlegen

Du verwendest eine klare Ordnerstruktur. .h-Dateien enthalten Klassen-Deklarationen, .cpp-Dateien die Implementierung.

Dateibaum im ZIP

```
projekt/
  include/Player.h
  include/Board.h
  include/Game.h
  src/main.cpp
  src/Board.cpp
  src/Game.cpp
  uebungen/01_hallo.cpp
  uebungen/02_board_test.cpp
  tests/board_tests.cpp
  data/               (wird fuer Spielstaende benutzt)
  docs/TESTPLAN.md
  README.md
```

Deine Schritte

- ZIP entpacken und in VS Code über „Datei > Ordner öffnen“ den Ordner projekt auswählen.
- Links im Explorer die Ordner include, src, uebungen und tests finden.
- Das Terminal über der richtigen Ordner-Ebene öffnen: src und include müssen im aktuellen Pfad liegen.
- Erklären können: main.cpp startet das Programm, Board.cpp regelt das Feld, Game.cpp organisiert das Spiel.

> Warum trennen? Beim objektorientierten Programmieren bekommt jeder Bereich eine klar erkennbare Verantwortung. Das reduziert doppelten Code und erleichtert Tests.

03 / PROGRAMMIER-ANLEITUNG

Erstes C++-Programm kompilieren

Bevor du Klassen programmierst, stellst du sicher, dass das Übersetzen und Ausführen funktioniert.

Datei uebungen/01_hallo.cpp

```
#include <iostream>

int main() {
    std::cout << "Vier gewinnt - Gym Edition!\n";
    return 0;
}
```

Unter Windows im Projektordner eingeben

```
g++ -std=c++17 uebungen/01_hallo.cpp -o hallo.exe
.\hallo.exe
```

Soll-Ausgabe

```
Vier gewinnt - Gym Edition!
```

Erklärung in einfachen Worten

- #include <iostream> bindet die Konsolenausgabe ein.
- main() ist der Startpunkt des Programms.
- std::cout schreibt Text in das Terminal.
- return 0 bedeutet: ohne Programmfehler beendet.

Kontrolle: Wenn du jetzt diesen kleinen Schritt erfolgreich starten kannst, ist deine C++-Umgebung funktionsfähig. Erst dann mit Klassen weitermachen.

04 / PROGRAMMIER-ANLEITUNG

Klasse Player verstehen

Eine Klasse ist ein Bauplan. Ein Objekt ist ein konkreter Spieler, z. B. „Anna“ mit dem Spielsymbol R.

Datei: include/Player.h

```
#pragma once
#include <string>

class Player {
private:
    std::string name_;
    char symbol_;
public:
    Player(std::string name, char symbol)
        : name_(name), symbol_(symbol) {}
    const std::string& getName() const { return name_; }
    char getSymbol() const { return symbol_; }
};
```

Was bedeutet das?

- private: Niemand außerhalb der Klasse ändert den Namen versehentlich direkt.
- public: Auf diese Funktionen darf das Spiel zugreifen.
- Player(...): Konstruktor. Er gibt dem Spieler beim Erstellen Name und Symbol.
- getName() und getSymbol(): Getter, sie liefern Werte zurück.

Mini-Aufgabe: Erstelle gedanklich Player("Anna", 'R'). Welchen Wert liefert getName()? Und was würde getSymbol() liefern?

05 / PROGRAMMIER-ANLEITUNG

Spielfeld mit 2 bis 4 Spielern aufbauen

Mit einem zweidimensionalen std::vector bleibt die Spiellogik dieselbe. Nur die Größe verändert sich.

Vorgabe aus Aufgabenblatt 51

Spieler	Spalten	Reihen
2	7	6
3	9	8
4	11	10

Dateien: include/Board.h und src/Board.cpp

```
Board::Board(int playerCount)
  : rows_(6 + 2 * (playerCount - 2)),
    columns_(7 + 2 * (playerCount - 2)),
    grid_(rows_, std::vector<char>(columns_, '.')) {}
```

So rechnest du die Größe

Für 4 Spieler ist der Zuschlag 2 × (4 − 2) = 4. Damit entstehen 11 Spalten und 10 Reihen. Der Punkt . kennzeichnet ein noch freies Feld.

Kontrolle: Warum wird die Spielfeldgröße im Konstruktor berechnet und nicht für jede Spielerzahl als komplett neues Spiel programmiert?

06 / PROGRAMMIER-ANLEITUNG

Kurzhanteln in eine Spalte fallen lassen

Der Spieler wählt eine Spalte. Das Programm sucht von unten nach oben das erste freie Feld.

Kernprinzip: Schwerkraft

```
int Board::nextFreeRow(int col) const {
    if (col < 0 || col >= columns_) return -1;
    for (int row = rows_ - 1; row >= 0; --row) {
        if (grid_[row][col] == '.') return row;
    }
    return -1; // Spalte ist voll
}
```

Stein wirklich setzen

```
bool Board::drop(int column, char symbol, int& rowPlaced) {
    rowPlaced = nextFreeRow(column);
    if (rowPlaced == -1) return false;
    grid_[rowPlaced][column] = symbol;
    return true;
}
```

Wichtiger Unterschied: Der Benutzer sieht Spaltennummern 1, 2, 3 … Im Vector beginnt die Zählung bei 0. Deshalb wird vor dem Aufruf aus Eingabe 4 der Index 3.

07 / PROGRAMMIER-ANLEITUNG

Kleine Übung mit dem Board testen

Du musst nicht sofort das ganze Spiel verstehen. Teste zuerst nur eine einzige gesetzte Kurzhantel.

Datei: uebungen/02_board_test.cpp

```
#include "Board.h"
#include <iostream>

int main() {
    Board board(2);
    int row = -1;
    board.drop(3, 'R', row);
    board.print();
}
```

Übersetzen und starten

```
g++ -std=c++17 -Iinclude `
    uebungen/02_board_test.cpp src/Board.cpp `
    -o board-demo.exe
.\board-demo.exe
```

Der Stein R steht in der vierten Spalte ganz unten. Die Funktion print() stellt Spielsteine als =[R]= oder =[B]= dar. Das ist die einfache ASCII-Version einer Kurzhantel.

Übungsaufgabe

- Ändere Board board(2) zu Board board(4). Prüfe die größere Fläche.
- Ändere drop(3, ...) auf drop(0, ...). In welcher Spalte landet der Stein?
- Setze mehrfach in dieselbe Spalte. Wie verändert sich die Reihe?

08 / PROGRAMMIER-ANLEITUNG

Gewinnprüfung: vier in einer Reihe

Es reicht, vom zuletzt gesetzten Stein aus in vier Richtungen zu prüfen.

Die vier Richtungsvektoren

Typ	Zeilenrichtung	Spaltenrichtung
Waagrecht	0	+1
Senkrecht	+1	0
Diagonal ↘	+1	+1
Diagonal ↙	+1	−1

Ablauf in Board::hasWon(...)

```
const int dr[4] = {0, 1, 1, 1};
const int dc[4] = {1, 0, 1, -1};
for (int i = 0; i < 4; ++i) {
    int len = 1
        + countDirection(row, col, dr[i], dc[i], symbol)
        + countDirection(row, col, -dr[i], -dc[i], symbol);
    if (len >= 4) return true;
}
return false;
```

Warum plus Gegenrichtung? Der zuletzt gesetzte Stein kann mitten in einer Viererkette liegen. Deshalb zählst du denselben Spielertyp in beiden Richtungen und addierst den Ausgangsstein einmal.

09 / PROGRAMMIER-ANLEITUNG

Hauptmenü und Spielschleife programmieren

Game verbindet Spieler, Spielfeld und Eingabe. Das Menü und die Partie haben jeweils eigene Schleifen.

Die Aufgaben der Klasse Game

- run(): Menüpunkte 0–6 anzeigen und Auswahl auswerten.
- newGame(): Namen prüfen, Spieler erzeugen, Feld leeren.
- play(): abwechselnd gültige Züge verarbeiten.
- showHelp(): Spielregeln und Sonderbefehle anzeigen.

Ausschnitt aus src/main.cpp

```
#include "Game.h"

int main() {
    Game game;
    game.run();
    return 0;
}
```

Spielzug ganz vereinfacht

```
Eingabe lesen
-> Spalte pruefen
-> Kurzhantel setzen
-> Gewinnpruefung
-> bei keinem Gewinn: naechsten Spieler waehlen
```

Objektorientierung: main() soll nicht tausend Zeilen Spiellogik enthalten. Game verwaltet den Ablauf; Board entscheidet, was auf dem Feld passiert.

10 / PROGRAMMIER-ANLEITUNG

Spieleranzahl, Namen und Zugwechsel

Die Spielerzahl darf nur 2, 3 oder 4 sein. Spieler erhalten feste, verschiedene Symbole.

Spieler und Symbole

Spieler	Symbol	ASCII-Kurzhantel
1	R	=[R]=
2	B	=[B]=
3	G	=[G]=
4	Y	=[Y]=

Warum funktioniert das Wechseln für vier Spieler?

```
current_ = (current_ + 1)
           % static_cast<int>(players_.size());
```

Bei 3 Spielern ergibt die Formel 0 → 1 → 2 → 0. Das Modulo-Zeichen % liefert den Rest der Division, damit der Index nach dem letzten Spieler wieder bei 0 beginnt.

Eingabeprüfung

- Eine Zahl kleiner 2 oder größer 4 wird abgelehnt.
- Leere und doppelte Namen werden erneut angefordert.
- Bei falscher Spalte oder voller Spalte darf derselbe Spieler noch einmal wählen.

> Test: Wähle 4 Spieler und prüfe 11 × 10 Felder. Gib danach einen Buchstaben statt einer Menüzahl ein. Das Programm soll nicht abstürzen.

11 / PROGRAMMIER-ANLEITUNG

Sonderbefehle in der laufenden Partie

Die Zusatzfunktionen sind Teil der Eingabeschleife und müssen den Spielzustand sauber behandeln.

Eingabe	Aktion	Wichtige Regel
V 4	Zugvorschau Spalte 4	Kein echter Zug, kein Zugwechsel
U	Letzten Zug zurück	Aktueller Spieler wird zurückgesetzt
S	Spielstand speichern	Keine Veränderung des Spielfelds
I	Spielfeldinformation	Nur Anzeige, kein Zugwechsel
H	Hilfe anzeigen	Nur Anzeige, kein Zugwechsel
A	Animation umschalten	Bei ANSI-Problemen abschalten
M	Zurück ins Hauptmenü	Aktive Partie bleibt im Speicher

So setzt du die Vorschau um

Board::print(previewColumn, symbol) zeigt das Symbol probeweise am nächsten freien Ort. Das Feld wird dabei nicht überschrieben. Deshalb hat V 4 keinen Einfluss auf späteres Spielen.

Rückgängig ohne Fehler

history_ speichert für jeden gültigen Zug die Reihe, Spalte und den Spielerindex. U nimmt den letzten Eintrag zurück und entfernt den obersten Stein aus der betreffenden Spalte.

> Zusatzregel: Speichern, Hilfe, Vorschau und Info zählen nicht als Spielzüge. Auch eine ungültige Zahl darf nicht zum nächsten Spieler wechseln.

12 / PROGRAMMIER-ANLEITUNG

Fallanimation und Gaming-Darstellung

Die Referenz funktioniert in der Konsole. Die 3D-Bilder sind ein separater Gestaltungsentwurf.

Bereits im C++-Lernprojekt enthalten

- Kurzhantel-Symbole als Text: =[R]=, =[B]=, =[G]= und =[Y]=.
- Kleine Konsolenanimation: Board::animateDrop() zeichnet den Stein zeilenweise weiter unten.
- Mit A lässt sich die Animation ein- oder ausschalten.

So arbeitet die Animation

```
for (int row = 0; row <= landingRow; ++row) {
    // Konsole leeren und Spielfeld neu zeichnen
    // Symbol nur fuer diese Anzeige an row/column setzen
    std::this_thread::sleep_for(
        std::chrono::milliseconds(65));
}
// Erst DANACH wird der echte Stein im Board gespeichert.
```

Was später für die echte 2D/3D-Gaming-UI nötig ist

- Eine Grafikbibliothek wie SFML oder SDL2 und ein Renderfenster.
- Kurzhantel-Sprites bzw. gezeichnete Rechtecke statt Konsolentext.
- Ein zeitabhängiger Fall-Übergang bis zur berechneten Zielreihe.
- Maus- oder Tastatursteuerung, die den vorhandenen Board-Aufruf benutzt.

> Abgrenzung: Die Start-/Hilfe-Screens in der PDF sind Mockups. Eine grafische Fenster-Oberfläche gehört nicht zur bereits getesteten C++-Konsolenversion.

13 / PROGRAMMIER-ANLEITUNG

Spielstand speichern und laden

Speichern bedeutet: genug Informationen ablegen, um dieselbe Partie später fehlerfrei fortzusetzen.

Das Lernprojekt speichert in data/savegame.txt

```
GYM1
2 1 6 7 1
"Anna" R
"Ben" B
5 3 0
.......
.......
.......
.......
.......
...R...
```

Erste Zeile: Dateiformat-Version. Zweite Zeile: Spielerzahl, nächster Spieler, Reihen, Spalten und Zahl der Züge. Danach folgen Namen, Symbole, Zughistorie und alle Feldzellen.

Warum beim Laden prüfen?

- Spielerzahl und Maße müssen zusammenpassen.
- Ein Zug darf nur in einer tatsächlich freien Spalte landen.
- Der rekonstruierte Spielverlauf muss mit den gespeicherten Zellen übereinstimmen.
- Erst nach vollständiger Prüfung ersetzt der geladene Zustand die laufende Partie.

> Test: Spiele einen Zug, gib S ein, gehe mit M ins Menü und wähle 5. Fehlt die Datei oder ist sie beschädigt, folgt eine Meldung statt eines Absturzes.

14 / PROGRAMMIER-ANLEITUNG

Highscore dauerhaft speichern

Gewonnene Spiele werden je Spielername erfasst und beim Anzeigen nach Siegen sortiert.

C++-Datenstruktur

```
std::map<std::string, int> highscores_;

// Nach einem gueltigen Sieg:
++highscores_[name];
saveHighscores();
```

Wie sieht die Anzeige aus?

Platz	Spieler	Siege
1	Anna	8
2	Ben	6
3	Clara	4

Zuerst nach Siegen absteigend sortieren. Bei gleicher Zahl alphabetisch nach Name. Die Datei liegt im Ordner data/highscore.txt.

Worauf du achten musst

- Siege nur erhöhen, nachdem der Gewinn korrekt erkannt wurde.
- Ein einmal gewonnener Spielstand darf nicht noch einmal als neue Partie zählen.
- Nach Programmneustart muss die Datei wieder geladen werden.

Kontrolle: Gewinne eine Partie, beende das Programm, starte es erneut und wähle Menüpunkt 6. Bleibt der Sieg erhalten?

15 / PROGRAMMIER-ANLEITUNG

Das ganze Projekt starten

Wenn die einzelnen Bausteine verständlich sind, baust du sie zu einer vollständigen Anwendung zusammen.

Windows: VS-Code-Terminal im Projektordner

```
g++ -std=c++17 -Wall -Wextra -pedantic -Iinclude `
  src/main.cpp src/Board.cpp src/Game.cpp `
  -o vier-gewinnt.exe
.\vier-gewinnt.exe
```

Linux / WSL: Terminal im Projektordner

```
g++ -std=c++17 -Wall -Wextra -pedantic -Iinclude \
  src/main.cpp src/Board.cpp src/Game.cpp \
  -o vier-gewinnt
./vier-gewinnt
```

Schnelltest nach dem Start

- Menü 2: Spielerzahl wählen, beispielsweise 2.
- Menü 3: Namen eingeben, beispielsweise Anna und Ben.
- Menü 1: Partie starten.
- Im Spiel V 4 für Vorschau, 4 für echten Stein, U zum Zurücknehmen, S zum Speichern.
- M zum Menü, 5 zum Laden, 6 für den Highscore.

> Die Reihenfolge ist wichtig: Zuerst kompilieren, dann ausführen. Wenn du Quelldateien änderst, musst du neu kompilieren, damit die Änderungen in der .exe stehen.

16 / PROGRAMMIER-ANLEITUNG

Automatische und manuelle Tests

Prüfe Regeln einzeln. Das erspart dir Fehlersuche während der Vorführung.

Testdatei: tests/board_tests.cpp

```
g++ -std=c++17 -Wall -Wextra -pedantic -Iinclude `
  tests/board_tests.cpp src/Board.cpp -o board-tests.exe
.\board-tests.exe
```

Die bereitgestellten Board-Tests prüfen unter anderem alle Spielfeldgrößen, ungültige Spalten, waagerechte, senkrechte und diagonale Gewinnlinien sowie Undo.

Weitere manuelle Abnahmetests

Test	Erwartetes Ergebnis
V 4	Spielfeld bleibt unverändert
Spalte voll	Ablehnen, gleicher Spieler bleibt am Zug
S und anschließend Laden	Identischer Spielzustand
Beschädigte Save-Datei	Fehlermeldung ohne Überschreiben
Gewinn und Neustart	Sieg bleibt im Highscore erhalten
A	Animation wird ein- bzw. ausgeschaltet

Für die Abgabe: In docs/TESTPLAN.md eigene Ist-Ergebnisse, Datum und Belege ergänzen. Diese Anleitung enthält nicht den Nachweis, dass alle manuellen Tests bereits durchgeführt wurden.

17 / PROGRAMMIER-ANLEITUNG

Häufige Fehlermeldungen lösen

Die folgenden Probleme lassen sich meist ohne große Änderungen beheben.

Fehlermeldung	Ursache / Lösung
g++ not recognized	Compiler fehlt oder ist nicht im PATH. Terminal nach Einrichtung neu öffnen.
Board.h: No such file	Aus Projektordner arbeiten und den Parameter -Iinclude setzen.
undefined reference to Board::...	Beim Kompilieren src/Board.cpp zusätzlich angeben.
undefined reference to Game::...	src/Game.cpp wurde beim Bauen vergessen.
No such file: src/main.cpp	Terminal befindet sich im falschen Ordner.
Datei beim Laden fehlt	Aus Projektordner starten und data/savegame.txt prüfen.
Komische Sonderzeichen	Konsole unterstützt ANSI eventuell nicht. Mit A Animation abschalten.
exe wird nicht aktualisiert	Nach Änderungen erneut g++ ausführen.

Einfacher Diagnoseablauf

- 1. Die erste Compiler-Fehlermeldung lesen, nicht die letzte.
- 2. Dateiname und Zeilennummer öffnen.
- 3. Klammern, Semikolons, #include und Schreibweise prüfen.
- 4. Erneut kompilieren, erst dann testen.

Schulregel: Fehlermeldungen verstehen statt wahllos den Code zu ersetzen. Notiere eine Ursache und eine Lösung in deinem Entwicklungsprotokoll.

18 / PROGRAMMIER-ANLEITUNG

GitHub und Projektdokumentation

Dein Lehrer soll nachvollziehen können, wie die Lösung schrittweise entstanden ist.

Eine sinnvolle Commit-Historie

```
git init
git add .
git commit -m "docs: Projektstruktur und Anforderungen"
git add include/Board.h src/Board.cpp
git commit -m "feat: dynamisches Spielfeld"
git add include/Game.h src/Game.cpp
git commit -m "feat: Menu und Spielablauf"
```

Welche Dokumente gehören dazu?

- README.md: Idee, Voraussetzungen, Startanweisung, Klassenüberblick.
- docs/TESTPLAN.md: Testfälle, Ist-Ergebnisse und offene Fehler.
- docs/pap/: Ablaufdiagramme P01 bis P12.
- docs/prompts.md: verwendete KI-Fragen und eigene fachliche Überarbeitung.
- Sprechfähige kurze Commits nach jedem überprüften Zwischenstand.

Copyright: Die Design-ID JK-CD-DE-2026-003 verweist auf dein Gestaltungssystem. Lehrer-Aufgabenblatt, eigene Arbeit und KI-unterstützte Bausteine müssen fachlich korrekt unterschieden werden.

19 / PROGRAMMIER-ANLEITUNG

Vom Konsolenprogramm zur Gaming-UI

Baue die echte grafische Oberfläche erst auf die funktionierende Spiellogik auf.

Vier Ebenen, damit der Code übersichtlich bleibt

Ebene	Aufgabe
Model (Board, Player)	Felder, Regeln, Spieler - kennt keine Grafik
Controller (Game)	Eingabe auswerten, Spieler wechseln, Sieg melden
Renderer (später)	Kurzhantel-Sprites, Hintergrund und Animation zeichnen
Storage	Spielstand und Highscore in Dateien verwalten

Schrittweise Erweiterung

- 1. Kurzhantel-Grafiken zeichnen oder als eigene Bilddateien vorbereiten.
- 2. Ein einfaches Fenster öffnen und ein leeres Raster anzeigen.
- 3. Mausklick auf eine Spalte in einen Board::drop-Aufruf umwandeln.
- 4. Die fallende Kurzhantel für mehrere Frames zwischen Start und Ziel zeichnen.
- 5. Menüs, Highscore und Hilfe als eigene Bildschirme ergänzen.

Technische Grenze: Die bisherigen UI-Bilder sind Konzeptgrafiken. Die Code-ZIP enthält kein SFML-/SDL2-Projekt und keine klickbare Gaming-Oberfläche. Das Konsolenspiel ist der erste umsetzbare Schritt.

20 / PROGRAMMIER-ANLEITUNG

Welche Funktionen gehören zu welchem PAP?

Diese Zuordnung hilft beim Übergang von Ablaufdiagrammen zu echten Klassenmethoden.

PAP	Codebereich
P01 Hauptmenü	Game::run()
P02/03 Namen und Feld	Game::configureNames(), Board::Board()
P04 Spielschleife	Game::play()
P05 Gewinnprüfung	Board::hasWon(), countDirection()
P06 Spielstand speichern	Game::saveGame()
P07 Spielstand laden	Game::loadGame()
P08 Highscore	Game::addWin(), showHighscores()
P09 Sonderbefehle	Befehlsauswertung in Game::play()
P10 Vorschau	Board::print(previewColumn, symbol)
P11 Undo	Board::undo(), history_
P12 Match Insights	Game::showInfo()

Präsentationstipp: Erst einen Ablaufplan zeigen, dann genau eine zugehörige Funktion im Code öffnen und mit einem konkreten Spielzug erklären. So vermeidest du unnötige Fragen zu unverbundenen Diagrammen.

21 / PROGRAMMIER-ANLEITUNG

Typische Lehrerfragen, direkt beantwortet

Du solltest die Antworten in eigenen Worten erklären können, nicht nur auswendig vorlesen.

Frage	Kurze technische Antwort
Warum OOP?	Die Klasse Board kapselt Spiellogik; Game steuert den Ablauf; Player speichert Spielerdaten.
Warum vector<vector<char>>?	Die Größe ist dynamisch und bleibt für 2–4 Spieler wiederverwendbar.
Warum 0-basierter Index?	C++-Vector beginnt bei 0; Benutzereingaben beginnen bei 1 und werden umgerechnet.
Wie wird ein voller Schacht erkannt?	nextFreeRow() findet keinen Punkt mehr und liefert -1.
Warum vier Richtungen?	Horizontal, vertikal und beide Diagonalen decken alle Gewinnlinien ab.
Warum in Gegenrichtung zählen?	Der zuletzt gesetzte Stein kann in der Mitte der Viererkette liegen.
Wie wird Undo möglich?	Eine Historie hält für jeden Zug Zeile, Spalte und Spieler fest.
Warum ist Save-Version sinnvoll?	GYM1 macht ein Dateiformat erkennbar und künftig erweiterbar.
Was passiert bei kaputter Datei?	loadGame() validiert erst einen temporären Spielstand. Erst dann wird er übernommen.
Wo ist die echte 3D-UI?	Derzeit Mockup. Die getestete Anwendung ist ein textbasiertes Konsolenspiel.

Vertiefung: Bei der Abschlusspräsentation kann dein Lehrer auch nach Compiler-Warnungen, Testbelegen, fehlerhaften Eingaben und dem eigenen Beitrag fragen. Halte dafür GitHub-Commits und Tests bereit.

22 / PROGRAMMIER-ANLEITUNG

Abgabe- und Lerncheckliste

Markiere diese Punkte erst, wenn du sie tatsächlich selbst kontrolliert hast.

- Ich kann g++ erklären und ein Minimalprogramm kompilieren.
- Ich kann Header und .cpp-Dateien unterscheiden.
- Ich kann Player, Board und Game im Code finden und erläutern.
- Ich kann erklären, wie ein Stein nach unten fällt.
- Ich kann die vier Richtungsvektoren und die Siegprüfung zeichnen.
- Ich kann einen gültigen, ungültigen und einen vollen-Spalte-Zug demonstrieren.
- Ich habe Speichern/Laden selbst geprüft und Fehlerfälle getestet.
- Ich habe die Anzeige für 2, 3 und 4 Spieler geprüft.
- Ich weiß, dass die 3D-Gym-UI nur ein Designentwurf ist.
- Ich habe Testbelege, GitHub-Commits und KI-Prompts gesammelt.

Dein nächster Schritt

Öffne die ZIP im VS-Code-Editor und arbeite zuerst Kapitel 1 bis 7 durch. Danach startest du den vollständigen Code und änderst kontrolliert eine Kleinigkeit, zum Beispiel den Menütext oder den Namen der Kurzhantel-Symbole.

Projektstand: Diese Ausgabe enthält einen lauffähigen und an ausgewählten Fällen geprüften C++-Konsolen-Referenzstand. Sie ist keine Zusicherung, dass alle Schulanforderungen, Sonderfälle oder die grafische UI vollständig abgenommen wurden.

INHALT

JULIAN KRAUSS

ITA · PROGRAMMIERTECHNIK · 2026/27

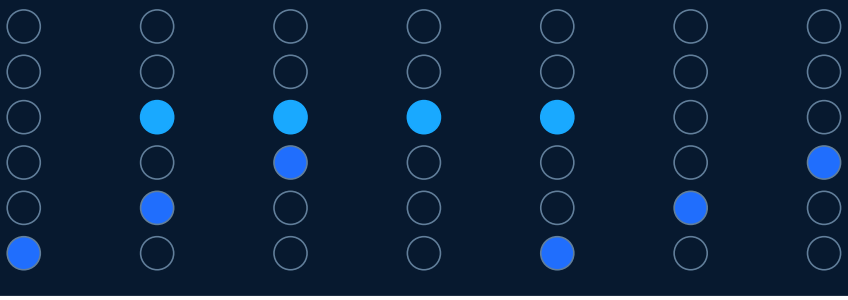
BERND BLINDOW GRUPPE
SCHULE AUSBILDUNG STUDIUM

PROJEKTAUFGABE 51 / C++ / OBJEKTORIENTIERT

VIER GEWINNT

PROGRAMMABLAUFPLÄNE

P01-P12 | Entwurf Edition 2.0 | Layout-Revision 3.1



KLARER ABLAUF. PRÜFBARE ENTSCHEIDUNGEN.

Vollständige technische Planung mit zwölf skalierbaren Ablaufdiagrammen, nachvollziehbarer C++-Architektur, geplanten Zusatzfunktionen und Testkonzept.

DOKUMENTATION / WERKDATEN

WERK-ID	JK-PT-2026-051-PAP
DESIGN-ID	JK-CD-DE-2026-003 · Rev. 3.2
UMFANG	12 PAPs / 38 Dokumentseiten
STATUS	Entwurf, keine bestätigte C++-Implementierung

© 2026 Julian Krauß · Mit dokumentierter KI-Unterstützung

Zielbild & Anforderungen

Fachliche Grundlage: Aufgabenblatt 51, Seiten 1 bis 4.

I V

Projektziel

Ein spielbares, konsequent objektorientiertes C++-Konsolenprogramm für zwei bis vier menschliche Spieler. Das Spielfeld wächst abhängig von der Spieleranzahl, während die Gewinnregel unverändert bleibt.

Verbindliche Kernfunktionen

- Hauptmenü mit den Auswahlwerten 0 bis 6 und nachvollziehbarer Benutzerführung.
- Spielernamen und Spieleranzahl erfassen; ungültige Eingaben abweisen.
- Gewinn in vier Richtungen, Unentschieden und korrekten Spielerwechsel erkennen.
- Spielzustand vollständig speichern/laden; fehlerhafte Dateien sicher behandeln.
- Highscore dauerhaft nach gewonnenen Spielen führen und absteigend anzeigen.

Skalierungsregel aus der Aufgabenstellung

Spieler	Spalten	Reihen	Viererfolge
2	7	6	4 in einer Linie
3	9	8	4 in einer Linie
4	11	10	4 in einer Linie

KLARE ABGRENZUNG

Die Pläne zeigen das vorgesehene Programmverhalten. Sie sind ein Architektur- und Umsetzungsentwurf, kein Nachweis eines bereits funktionsfähigen C++-Programms. Zusätzliche Bedienungsentscheidungen werden als solche gekennzeichnet.

So liest du den PAP

Jedes Hauptmodul erhält eine eigene Plan-ID und einen eindeutigen Rücksprung.

PAP-Register | Übersicht der acht Module

P01 Hauptmenü
Verzweigung auf alle Benutzeraktionen.

P02 Spielsteuerung
Spielstart, Schleife, Zugwechsel und Pause.

P03 Spielzug
Eingabevalidierung, volle Spalte, Setzen.

P04 Ergebnis
Gewinn, Remis oder Fortsetzung.

P05 Gewinnprüfung
Systematischer Test aller vier Richtungen.

P06 Speichern
Daten erfassen und sicher schreiben.

P07 Laden
Datei prüfen und Zustand wiederherstellen.

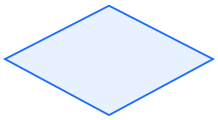
P08 Highscore
Sieg erfassen oder sortierte Liste zeigen.

Symbol-Legende | DIN-66001-orientiert


Start / Ende

Verarbeitung

Ein- / Ausgabe


Entscheidung

Leseregel: Pfeile zeigen den Kontrollfluss. Entscheidungen haben beschriftete Ausgänge (Ja / Nein beziehungsweise Menüwert). Doppelt umrandete Kästen verweisen auf einen separaten PAP.

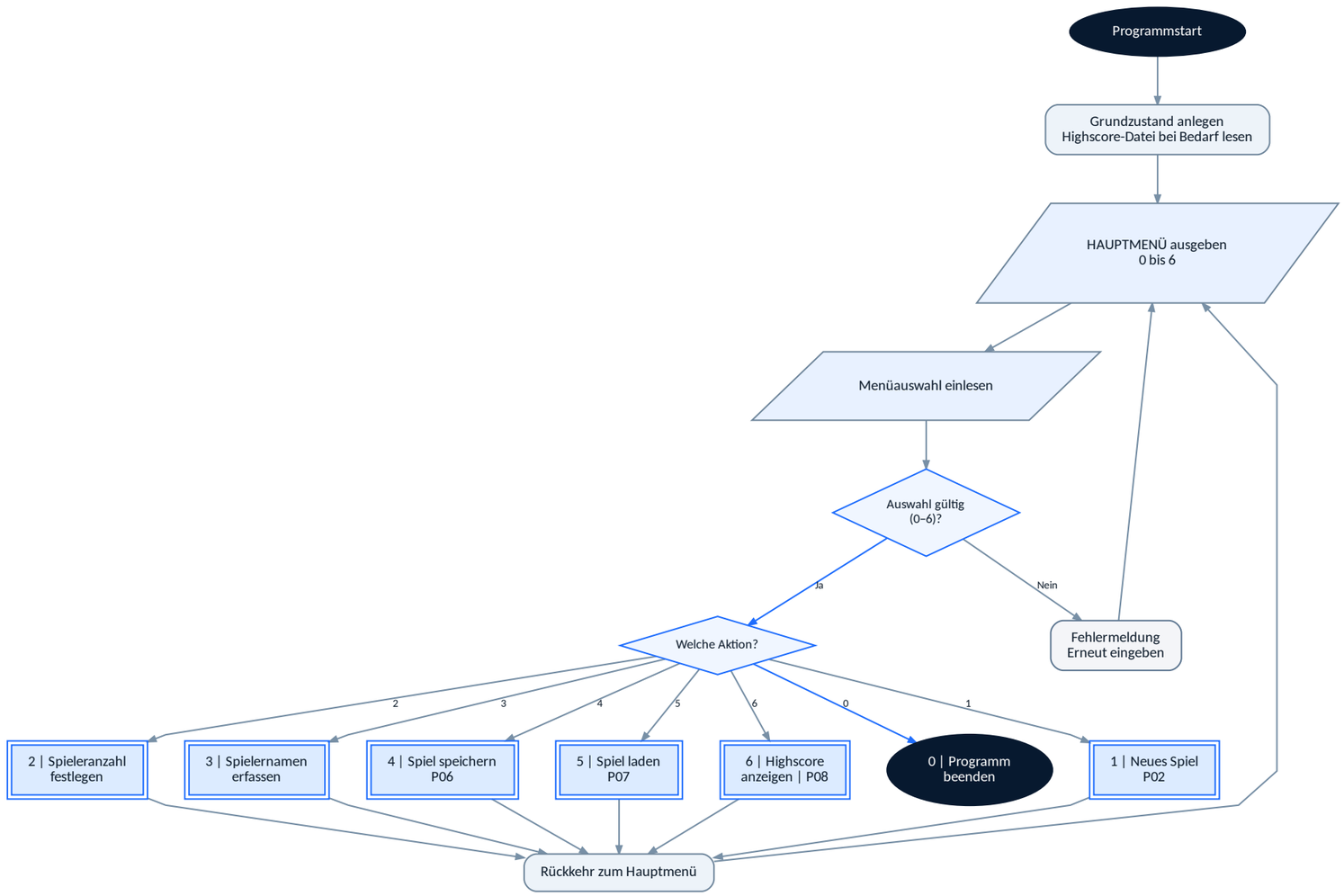
P01 / PROGRAMMFÜHRUNG

Hauptmenü

Die zentrale Navigation mit gültiger Auswahl, Fehlerpfad und Rücksprung.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / LESEN & ERKLÄREN

ZIEL

Das Menü ist die einzige Eintrittsstelle für Spielstart, Konfiguration und persistente Daten.

EINGABE

Menüeingabe 0 bis 6

ERGEBNIS

Aufruf einer Funktion oder sauberes Programmende

SICHERHEIT

Unbekannte Eingabe wird erneut abgefragt.

NACHWEIS

Ungültige Auswahl 7 bzw. Buchstaben führt nicht zum Absturz.

KURZABLAUF

01 Menü mit sechs Aktionen ausgeben

02 Auswahl einlesen und validieren

03 Gewählten Teilablauf starten

04 Zum Hauptmenü zurückkehren

© 2026 Julian Krauß | JK-PT-2026-051-PAP

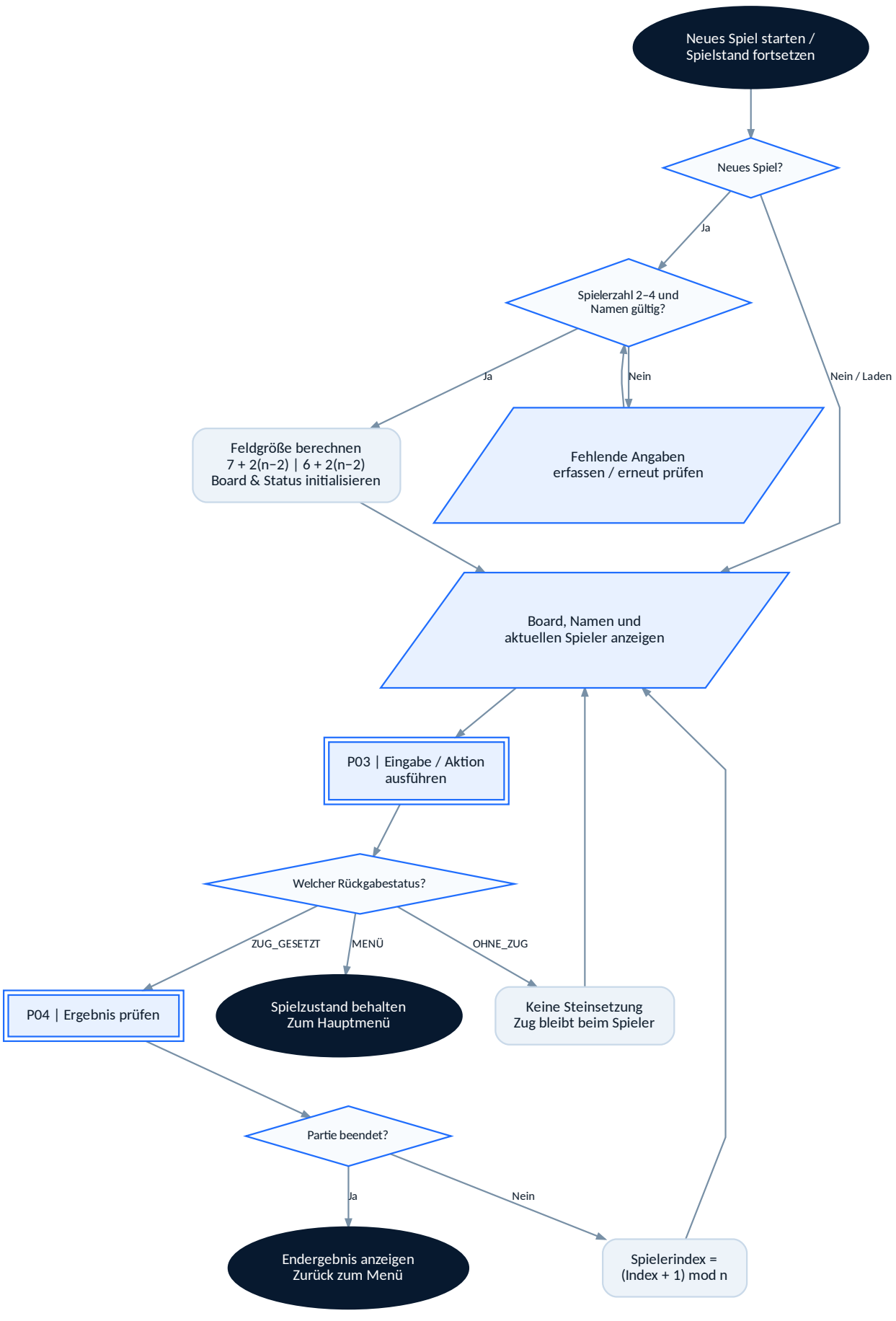
P02 / PARTIEVERWALTUNG

Spielsteuerung

Steuert die Spielphase vom Start oder Laden bis zum Ergebnis.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / LESEN & ERKLÄREN

ZIEL

Trennung von Eingabe, Brettregeln und Partiezustand verhindert doppelte Geschäftslogik.

EINGABE

Konfiguration oder geladener Spielstand

ERGEBNIS

Neuer Zug, Pause oder abgeschlossenes Spiel

SICHERHEIT

Zusatzbefehle verbrauchen keinen Spielzug.

NACHWEIS

Spielerwechsel nur nach gültigem Steinsetzen.

KURZABLAUF

01 Spielerangaben validieren

02 Board passend dimensionieren

03 Spielaktion an P03 weitergeben

04 Ergebnis P04 abfragen und Zustände wechseln

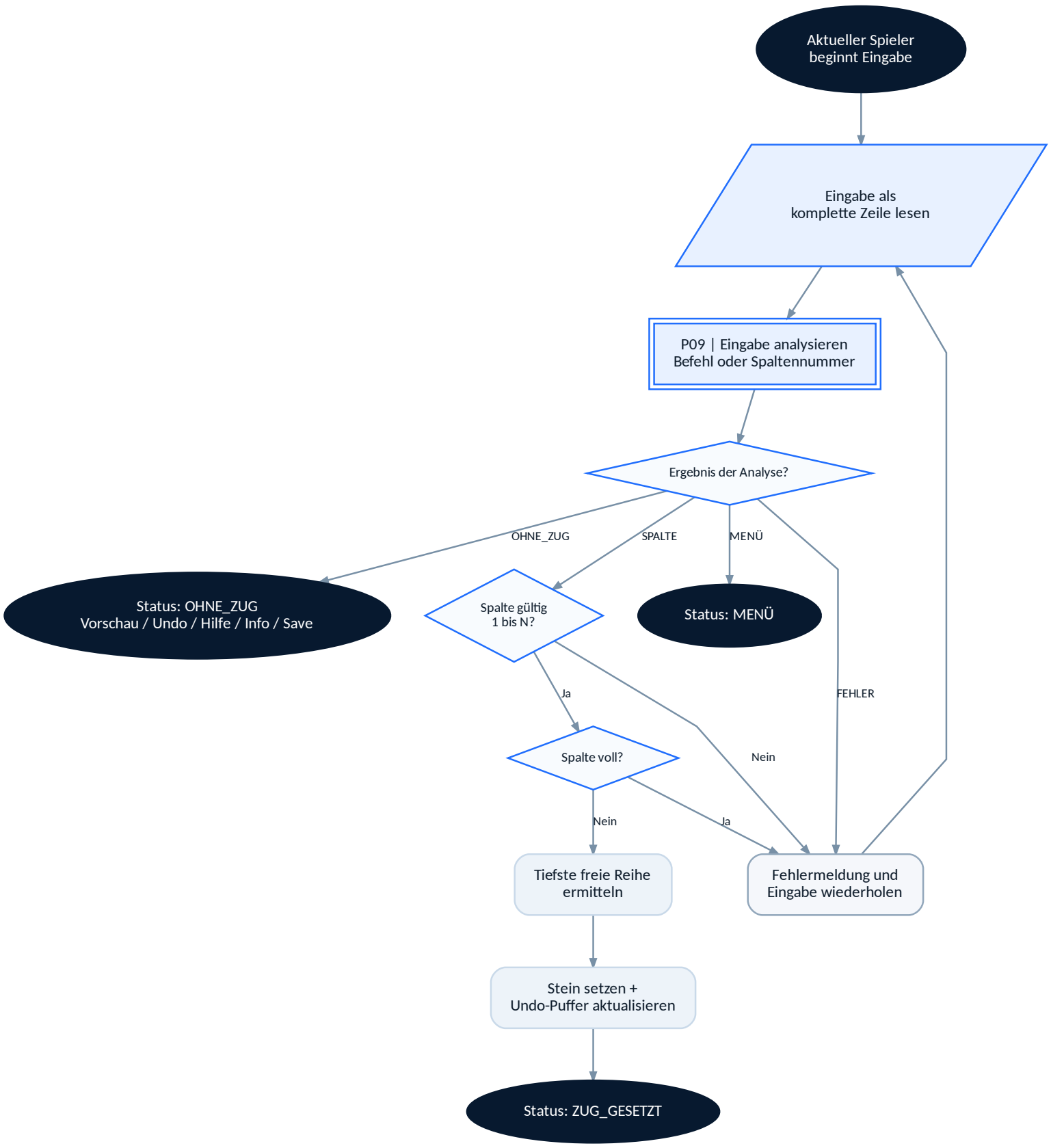
P03 / EINGABELOGIK

Spielzug und Validierung

Unterscheidet Menübefehle und gültige Zug-Eingaben.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / TECHNISCHE LESEHILFE

ZIEL

Eine einzige Eingaberoutine entscheidet über gültige Spalten und verhindert Zustandsfehler.

KONTROLLPUNKT

Volle Spalten, falsche Zahlen und ungültige Texte werden abgefangen.

ABNAHMETEST

Vorschau, Undo und Hilfe dürfen den Zug nicht verbrauchen.

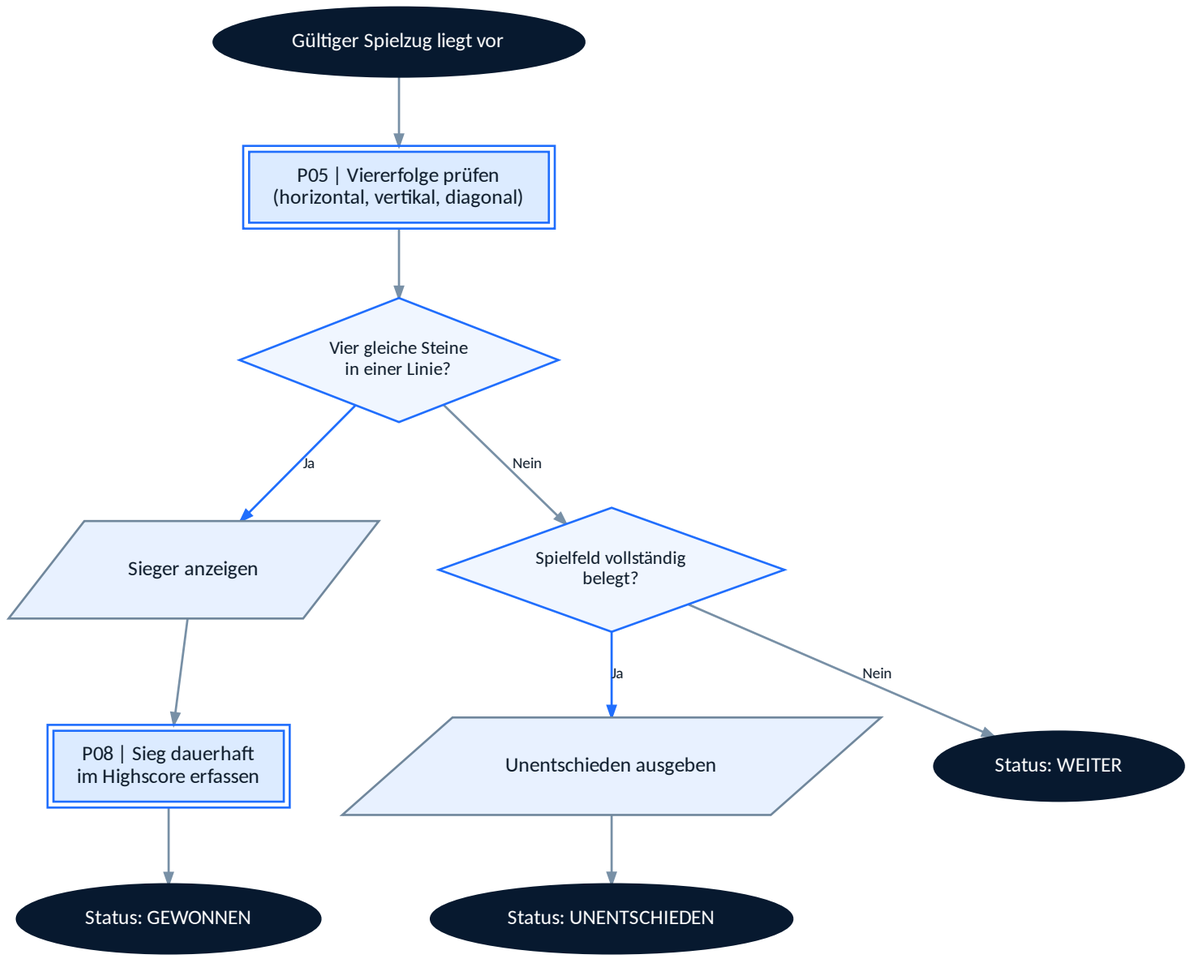
P04 / ENDZUSTAND

Spielergebnis

Unterscheidet Sieg, Unentschieden und Fortsetzung der Partie.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / LESEN & ERKLÄREN

ZIEL

Die Reihenfolge ist wichtig: Erst Gewinnprüfung, dann Remis und zuletzt Spielerwechsel.

EINGABE

Zuletzt gesetzter Stein

ERGEBNIS

Gewonnen, unentschieden oder weiterspielen

SICHERHEIT

Highscore höchstens einmal pro Gewinn aktualisieren.

NACHWEIS

Volles Feld mit Viererreihe bleibt ein Sieg, kein Remis.

KURZABLAUF

01 Gewinnprüfung aufrufen

02 Bei Sieg Ergebnis sichern

03 Nur danach volles Brett prüfen

04 Anderenfalls Partieverlauf fortsetzen

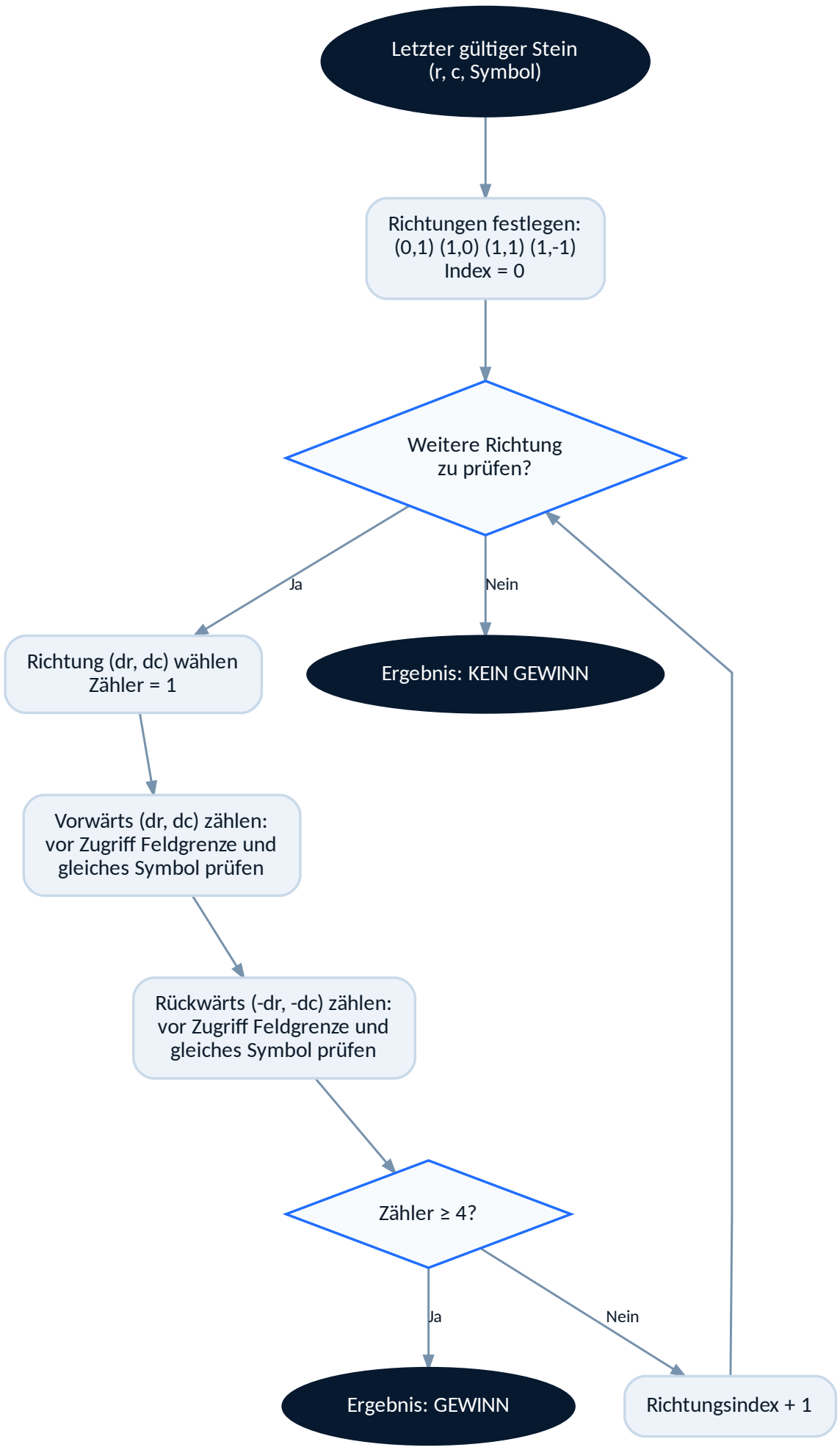
P05 / SPIELREGEL

Gewinnprüfung

Prüft vier verbundene Symbole horizontal, vertikal und diagonal.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / LESEN & ERKLÄREN

ZIEL

Die vier Gewinnrichtungen gelten unabhängig von der Spielfeldgröße.

EINGABE

Board, Symbol und letzter Zug

ERGEBNIS

true / false

SICHERHEIT

Rastergrenzen vor jedem Zellenzugriff prüfen.

NACHWEIS

Alle vier Richtungen und diagonale Randfälle abdecken.

KURZABLAUF

01 Jede Gewinnrichtung prüfen

02 Gleiche Symbole zusammenzählen

03 Rand oder Fremdsymbol stoppt

04 Ab vier verbundenen Steinen: Gewinn

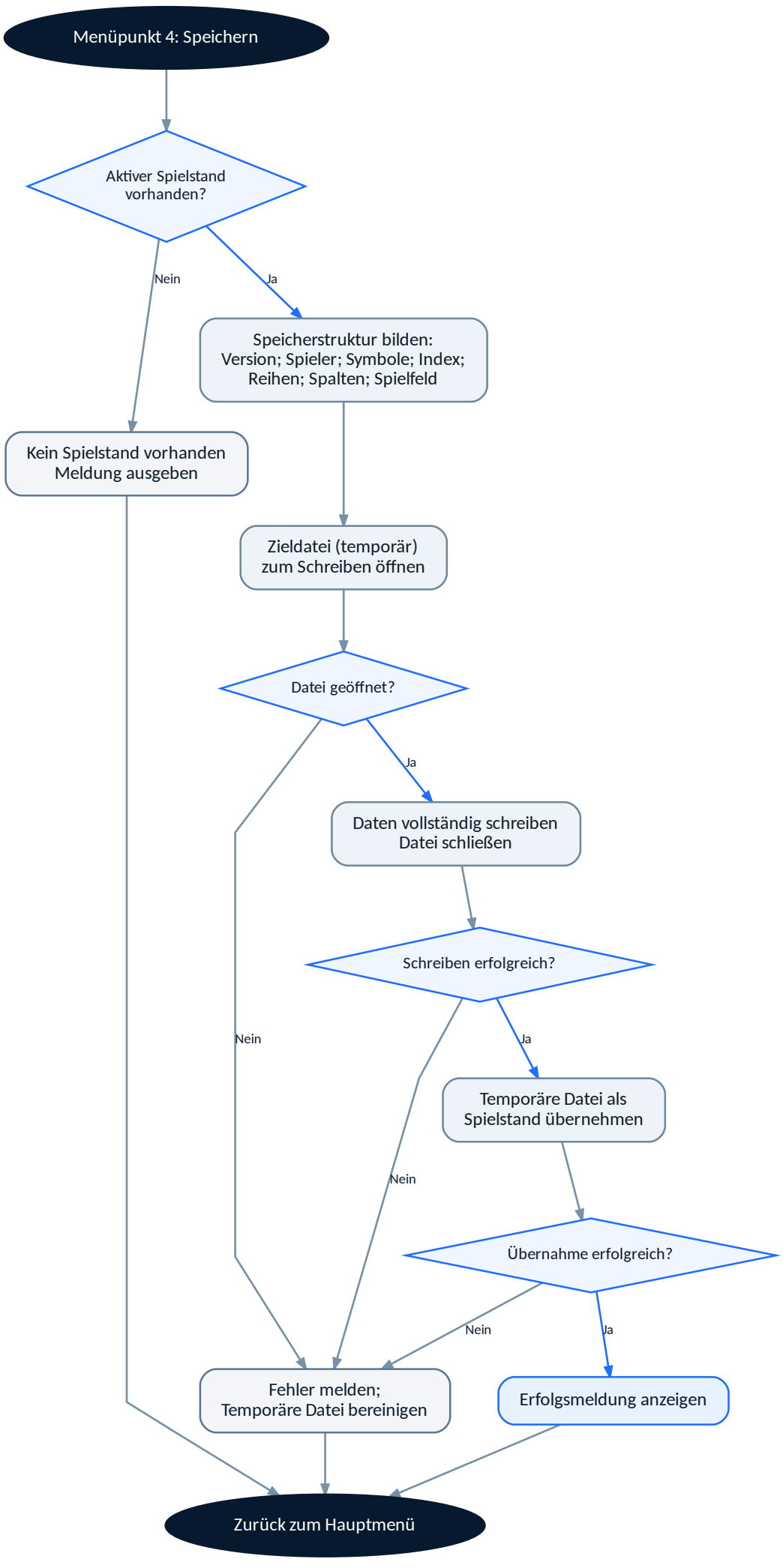
P06 / DATEIOPERATION

Speichern

Schreibt einen reproduzierbaren, vollständigen Spielzustand.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / LESEN & ERKLÄREN

ZIEL

Ein Spielstand muss nach Neustart dieselben Spieler, Dimensionen und aktiven Spieler erzeugen.

EINGABE

Gültiger Spielzustand

ERGEBNIS

Gespeicherte Datei oder Fehlermeldung

SICHERHEIT

Schreibfehler als Fehler behandeln und nicht als Erfolg melden.

NACHWEIS

Datei nach Neustart prüfen und wieder öffnen.

KURZABLAUF

01 Spieldaten erfassen

02 Datenformat samt Version erzeugen

03 Datei schreiben und prüfen

04 Ergebnis verständlich melden

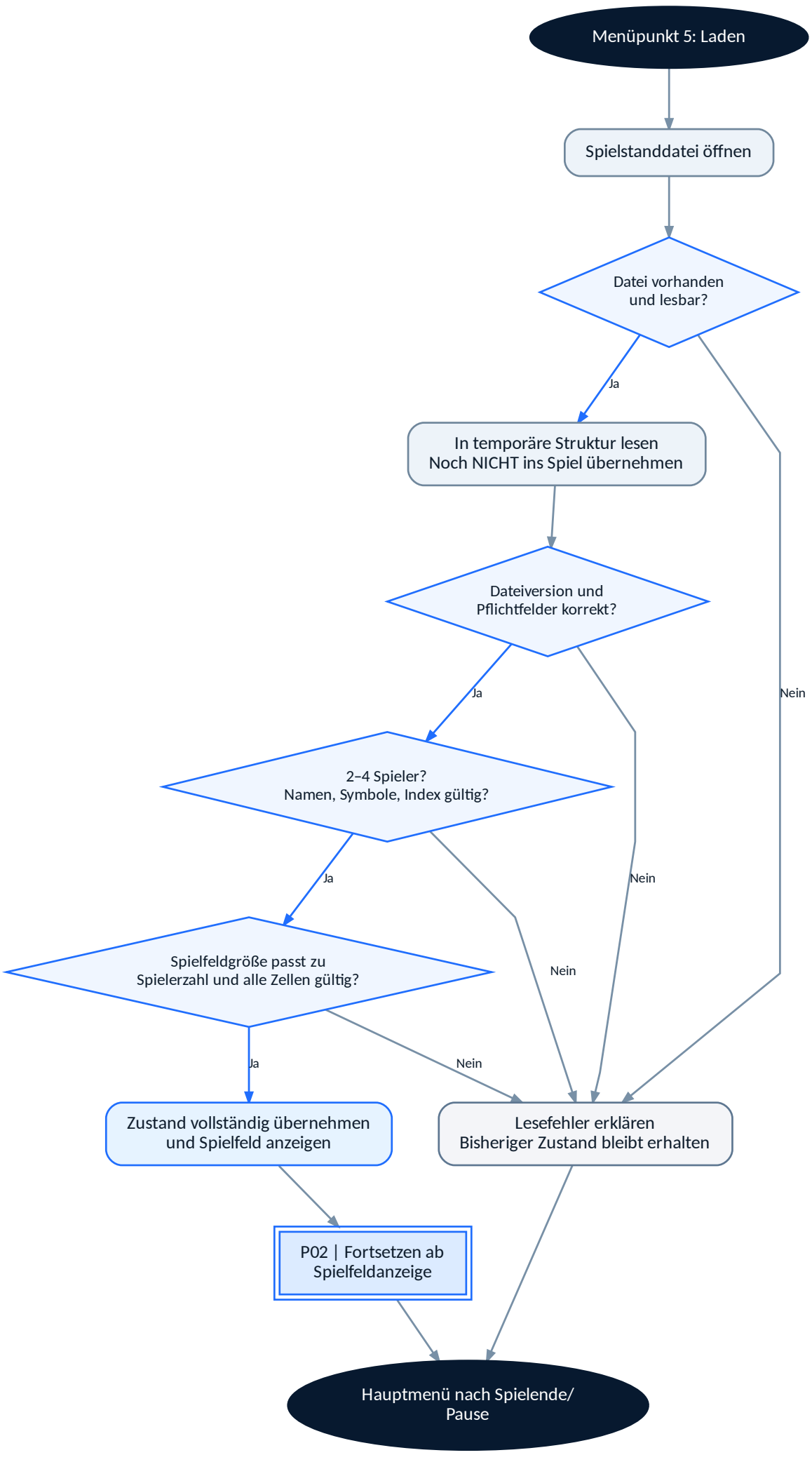
P07 / DATEISICHERHEIT

Laden und Prüfen

Übernimmt eine Datei erst nach vollständiger Validierung.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / LESEN & ERKLÄREN

ZIEL

Unvollständige oder beschädigte Dateien dürfen die laufende Partie nicht zerstören.

EINGABE

Dateipfad und Dateiinhalt

ERGEBNIS

Geprüfter Spielstand oder Fehlertext

SICHERHEIT

Erst in temporäre Struktur lesen, danach übernehmen.

NACHWEIS

Fehlende Datei, falsche Dimensionen, defektes Raster.

KURZABLAUF

01 Datei öffnen und Version prüfen

02 Alle Felder validieren

03 Spielzustand konsistent herstellen

04 Erst dann aktives Spiel ersetzen

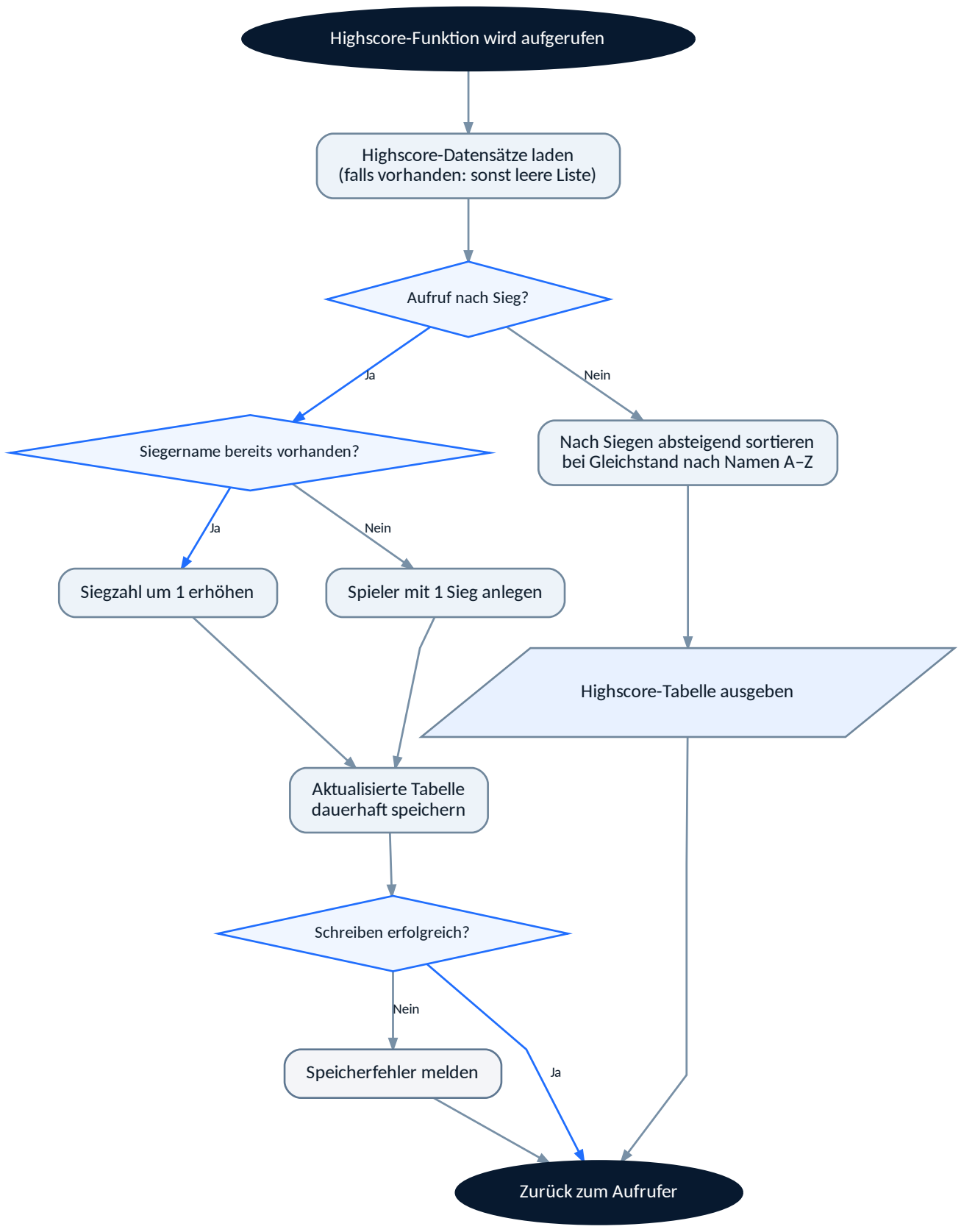
P08 / RANGLISTE

Highscore

Verwaltet Siege dauerhaft und sortiert absteigend.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / TECHNISCHE LESEHILFE

ZIEL

Die Rangliste ist vom einzelnen Spielstand getrennt.

KONTROLLPUNKT

Gleichstand nachvollziehbar alphabetisch ordnen.

ABNAHMETEST

Ein Sieg erhöht genau einen Datensatz um eins.

© 2026 Julian Krauß | JK-PT-2026-051-PAP

Objektorientiertes Konzept

Klassenentwurf: Responsibilities klar trennen, unnötige Abhängigkeiten vermeiden.

Vorgeschlagene Klassen und Aufgaben

Application / Menu

Zeigt Hauptmenü, prüft Auswahl, delegiert an Spiellogik und Verwaltungsfunktionen.

run(), showMenu(), readSelection()

Game

Hält Spieler, aktuelles Zugrecht und Status zusammen; steuert Spielschleife.

start(), continueGame(), nextPlayer()

Board

Speichert Raster und Abmessungen; verwaltet Einwurf, Gewinnprüfung und Vollstand.

placeToken(), checkWin(), isFull()

Player

Kapselt Namen und eindeutiges Symbol je Spieler.

getName(), getSymbol()

SaveManager

Schreibt/liest Spielstände, prüft Datenversion und Struktur vor Übernahme.

save(Game), load(Game&)

HighscoreManager

Verwaltet dauerhafte Siege pro Name, speichert und sortiert.

recordWin(), showRanking()

Bezug zur Vorlage: Player und Board sind in Aufgabenblatt 51 bereits als Beispielklassen skizziert. Application/Menu, Game, SaveManager und HighscoreManager sind ergänzende Architekturvorschläge für eine saubere Trennung der Zuständigkeiten.

Zustände, Regeln & Übergänge

Die nachfolgenden Regeln verbinden die PAP-Module zu einem konsistenten Ablauf.

Spielzustandsmodell

Zustand	Bedeutung / Auslöser
NICHT_GESTARTET	Es existiert noch kein Spielstand im Arbeitsspeicher.
AKTIV	Das Spielfeld ist gültig, und ein Spieler darf ziehen.
PAUSIERT	M führt ins Menü; das Spiel bleibt für Speichern/Laden erhalten.
GEWONNEN	Vier zusammenhängende Symbole erkannt; genau ein Sieg wird verbucht.
UNENTSCHIEDEN	Feld ist voll, ohne dass vorher ein Gewinn festgestellt wurde.

Zentrale Berechnungen

$$\text{Spalten} = 7 + 2 \times (\text{Spielerzahl} - 2)$$

$$\text{Reihen} = 6 + 2 \times (\text{Spielerzahl} - 2)$$

Spielerindex = (Spielerindex + 1) modulo Spielerzahl

Reihenfolge pro Zug: Eingabe prüfen → Stein in die tiefste freie Zelle setzen → Gewinn prüfen → bei Bedarf Unentschieden prüfen → nur sonst zum nächsten Spieler wechseln. Fehlerhafte Eingaben führen nicht zum Spielerwechsel.

Konkrete Designentscheidungen

Diese Festlegungen ergänzen die Aufgabe und sollten im Team bestätigt werden.

01 M während eines Spielzugs

Ein M unterbricht die laufende Partie und öffnet das Hauptmenü. Dadurch lässt sich Menüpunkt 4 „Speichern“ auch im laufenden Spiel verwenden.

02 Menüpunkt 1 „Neues Spiel“

Legt immer ein frisches Spielfeld an. Falls ein aktives Spiel besteht, wird vor dem Verwerfen eine Bestätigung abgefragt.

03 Menüpunkte 2 und 3

Ändern Spieleranzahl beziehungsweise Namen. Im aktiven Spiel werden Änderungen erst nach Bestätigung des Verwerfens zugelassen.

04 Menüpunkt 4 „Speichern“

Ist nur verfügbar, wenn sich eine Partie im Speicher befindet. Fehlt sie, gibt das Programm eine verständliche Meldung aus.

05 Menüpunkt 5 „Laden“

Lädt den geprüften Zustand und setzt das Spiel beim gespeicherten Spieler direkt fort. Bei Fehlern bleibt die vorige Partie erhalten.

06 Spielende

Kehrt nach Sieg oder Remis in das Hauptmenü zurück. Eine neue Partie wird erst auf ausdrücklichen Wunsch gestartet.

Speicherformat & Validierung

Mindestens geforderte Felder und ein fehlertoleranter Ladevorgang.

Beispiel eines zeilenorientierten Textformats

```
VG51|1
players=2
current=1
size=6x7
player=Anna|X
player=Ben|O
row=.......
row=.......
row=.......
row=.......
row=.......
row=X......
```

Lesart: Eine Spielstandversion, zwei Spieler, aktueller Spielerindex 1 (Ben), sechs Zeilen und sieben Spalten. Nur in der unteren linken Zelle liegt ein Stein von Anna. Beispielentwurf, kein vorgegebenes Originalformat.

Validierung vor der Übernahme

- Datei existiert, ist lesbar und enthält eine bekannte Versionskennung.
- Spieleranzahl liegt zwischen 2 und 4; alle Namen und Symbole sind gültig.
- Reihen/Spalten entsprechen exakt der vorgesehenen Skalierungsregel.
- Jede Rasterzelle enthält nur „leer“ oder ein zulässiges Spielersymbol.
- Spielerindex ist gültig; keine schwebenden Steine und keine unvollständigen Zeilen.

Wichtig: Namen mit Trennzeichen müssen beim Schreiben maskiert oder bei der Eingabe eingeschränkt werden. Dateioperationen melden Fehler; beim Laden werden Daten zuerst temporär geprüft und erst anschließend übernommen.

Highscore & Fehlerbehandlung

Die Wertung ist bewusst vom eigentlichen Spielfeld getrennt.

Siege korrekt verbuchen

Nach einem eindeutigen Gewinn wird der Datensatz des Siegers gesucht. Falls er existiert, erhöht sich seine Siegzahl exakt um eins. Andernfalls wird der Name mit einem Sieg angelegt. Anschließend werden die Werte dauerhaft gespeichert.

I V

Rangliste

Rang	Name	Siege
1	Mia	8
2	Leon	6
3	Noah	4
4	Emma	2

Das Beispiel stammt aus dem Aufgabenblatt. Standard: Siege absteigend; optional bei Gleichstand alphabetisch nach Name aufsteigend. Bei beschädigter Highscore-Datei wird eine verständliche Fehlermeldung ausgegeben.

Schutz vor typischen Fehlern

- Ein Sieg wird einmal gezählt. Ein abgeschlossenes Spiel darf beim späteren Laden keinen weiteren Sieg auslösen.
- Eine fehlerhafte Spielstanddatei darf keinen gültigen Zustand im Arbeitsspeicher überschreiben.
- Ein ungültiger Zug verändert weder das Raster noch den aktuellen Spieler.
- Ein fehlgeschlagener Highscore-Speichervorgang wird klar gemeldet und nicht als dauerhaft erfolgreich ausgegeben.

Tests & Abnahmekriterien

Jeder Test muss durch einen nachvollziehbaren Soll-Ist-Vergleich dokumentiert werden.

Funktionale Testfälle gemäß Aufgabenstellung

ID	Ausgangslage	Erwartetes Verhalten
T01	Menüwert -1 oder 9	Fehler, neue Abfrage
T02	Leerer Spielername	Name erneut anfordern
T03	Zug in volle Spalte	Kein Einwurf, gleicher Spieler
T04	Horizontal vier Steine	Sieg erkannt
T05	Vertikal vier Steine	Sieg erkannt
T06	Diagonal ↘ vier Steine	Sieg erkannt
T07	Diagonal ↙ vier Steine	Sieg erkannt
T08	Feld voll ohne Vierer	Unentschieden
T09	2/3/4 Spieler	7×6 / 9×8 / 11×10; Wechsel
T10	Speichern und Laden	Gleicher Zustand nach Neustart
T11	Datei fehlt / defekt	Fehlermeldung, kein Absturz
T12	Sieg & Gleichstand	+1; geordnete Rangliste

Abnahmeprinzip: Ein Test gilt nur dann als bestanden, wenn der tatsächliche Programmablauf mit dem erwarteten Ergebnis übereinstimmt. Das Ausführen der Tests ist noch Teil der Implementierung.

Zusätzliche Prüffälle

Gezielte Grenzwerttests ergänzen die zehn Abnahmepunkte der Vorlage.

ID	Testfall	Erwartung
T13	Buchstaben oder leer statt Spalte	Erneute Eingabe ohne Endlosschleife.
T14	Spalte 0 / größer als N	Keine Änderung am Spielstand.
T15	Spiel zu viert mit Zugfolge 0→1→2→3→0	Modulo-Wechsel bleibt korrekt.
T16	Spielername mit Leerzeichen	Speichern und Laden ohne Informationsverlust.
T17	Beschädigte Dimensionsangabe	Laden verweigert; alter Zustand unverändert.
T18	Sieger zweimal ausgewertet	Genau ein Highscore-Eintrag für eine Partie.

Checkliste vor jeder Demonstration

- Programm kompiliert ohne Fehler und relevante Warnungen.
- Zwei-Spieler-Grundversion mit drei Gewinnrichtungen und Remis getestet.
- Drei- und Vier-Spieler-Modus mit der richtigen Feldgröße geprüft.
- Speichern/Laden mit gültigen und ungültigen Dateien ausprobiert.
- Highscore aktualisiert sich korrekt, auch nach Neustart.
- Zentrale Methoden können ohne Ablesen erklärt werden.

Projektfahrplan 2026–2027

Monatliche Meilensteine aus dem Aufgabenblatt, kein behaupteter Fertigstellungsstand.

29.10.2026

Oktober

Anforderungen, Backlog, Repository, Klassenentwurf und Menü-Prototyp.

26.11.2026

November

Spielbare Zwei-Spieler-Grundversion ohne Speicherung.

17.12.2026

Dezember

Stabile Gewinn- und Remisprüfung, Fehlerbehandlung, Tests.

28.01.2027

Januar

Zwei bis vier Spieler, Save/Load und Highscore; Feature Freeze.

11.02.2027

Generalprobe

Live-Demo mit Zeitmessung vor der Präsentation.

18.02.2027

Vorführung

Präsentation, Anwendung, GitHub, Handout und Quellen.

Arbeitsweise: Donnerstags-Sprints mit Backlog, To Do, In Arbeit, Review/Test und Erledigt. Jede abgeschlossene Funktion wird getestet, kurz dokumentiert und in GitHub eingecheckt.

© 2026 Julian Krauß | JK-PT-2026-051-PAP

11 / RÜCKVERFOLGBARKEIT

Anforderung → PAP → Test

Jede Kernfunktion wird einem Ablaufplan und einem Test zugeordnet.

ANFORDERUNG	PAP	TEST-ID
Menüführung	**P01**	T01, T13
Spielernamen / Spielerzahl	**P01, P02**	T02, T09
Spielfeldgröße	**P02**	T09, T15
Gültige Eingabe / volle Spalte	**P03**	T03, T13, T14
Viererprüfung	**P04, P05**	T04–T07
Remis	**P04**	T08
Speichern	**P06**	T10, T16
Laden / Dateifehler	**P07**	T10, T11, T17
Highscore	**P08**	T12, T18

ABGABE UND REPOSITORY

Beispielhafte Ordnerstruktur für GitHub und Präsentation

vier-gewinnt-cpp/

src/	C++-Implementierung	**tests/**	Testfälle
include/	Headerdateien	**docs/pap/**	P01–P12 und Master-PDF
data/	Beispiel-Spielstände	**README.md**	Projektüberblick

DEFINITION OF DONE

Kompilierbar, getestet, sinnvoll kommentiert, auf GitHub eingecheckt und dokumentiert. Der PAP beschreibt die Planung. Funktionierende C++-Software und praktische Testnachweise bleiben erforderlich.

12 / DOKUMENTATION

Quellen, KI-Nutzung & Übergabe

Dokumentierte Grundlagen und offene Prüfungen bis zur C++-Abgabe.

PRIMÄRQUELLE

Dr. Michael Litera: Aufgabenblatt 51, Programmiertechnik, Projektaufgabe 2026/2027 (13 Seiten). Anforderungen und Menü: S. 2; Spielfeld und Klassen: S. 3–4; Termine: S. 6–7; Tests: S. 8; Bewertung und Fertigstellung: S. 12–13.

KI-NUTZUNG DOKUMENTIEREN

Nachvollziehbarer Arbeitsnachweis

DESIGN	Corporate Design Guide JK-CD-DE-2026-003, Rev. 3.2 / Release 4.6.3.
WERKZEUG	ChatGPT für Strukturierung, Formulierung und PAP-Diagramme.
AUFTRAG	Dokumentierten Ablaufplan für Vier gewinnt aus Aufgabenblatt 51 erstellen.
PRÜFPFLICHT	Alle Verzweigungen mit Aufgabenblatt und späterem C++-Code vergleichen.

BEWUSST ERGÄNZTE ANNAHMEN

- Eingabe M führt im laufenden Spiel zurück ins Hauptmenü.
- Beim Speichern wird eine temporäre Datei verwendet; Laden prüft den Zustand.
- Änderung der Spielerzahl verwirft ein aktives Spiel nur nach Bestätigung.
- Application/Menu, Game, SaveManager und HighscoreManager sind Klassenentwürfe.

VOR ABGABE PRÜFEN

Der PAP wird erst zur Ist-Dokumentation, wenn er mit dem tatsächlichen C++-Programm übereinstimmt. Änderungen gemeinsam versionieren sowie echte Testergebnisse und Screenshots nachtragen.

13 / ERWEITERUNG

Edition 2.0 | Feature-Upgrade

Kernregeln bleiben unverändert. Die zusätzlichen Funktionen sind optionale, klar priorisierte Entwurfsentscheidungen.

CORE FIRST. EXTRAS SECOND.

Die Kernanforderungen aus Aufgabenblatt 51 bleiben Pflicht. Erweiterungen werden erst freigegeben, wenn Zwei-Spieler-Spiel, Gewinnprüfung, Dateiverwaltung und Highscore funktionieren.

Vier besondere Erweiterungen und ein Komfortbefehl

01 **V n | Zugvorschau**
Zeigt, wo ein Stein fallen würde, ohne ihn zu setzen.

02 **U | Rückgängig**
Nimmt exakt den letzten gültigen Spielzug zurück.

03 **I | Match Insights**
Zeigt Spielstand, Zugzahlen und Spielfeld-Füllgrad.

04 **H | Schnellhilfe**
Erklärt Tasten und Eingabeformate kontextbezogen.

05 **S | Schnellspeichern**
Speichert den laufenden Zustand direkt aus dem Zugdialog.

Planstand: PAP und Bedienkonzept sind erstellt. Diese Erweiterungen sind noch keine bestätigte C++-Implementierung. Alle fünf Funktionen sind optional und werden erst nach stabiler Pflichtversion umgesetzt.

Pflicht und Zusatz trennen

Transparente Priorisierung verhindert, dass besondere Ideen die Projektbewertung gefährden.

Verpflichtend | Aufgabenblatt 51

Modul	Fachliche Leistung	Status
Hauptmenü	Auswahl 0–6, Fehleingaben	Geplant / PAP
Spieler & Board	2 bis 4 Spieler; 7×6, 9×8, 11×10	Geplant / PAP
Spiellogik	4er-Reihe in 4 Richtungen, Remis	Geplant / PAP
Speichern / Laden	Dateiprüfung und Wiederherstellung	Geplant / PAP
Highscore	Dauerhafte Siege, sortiert	Geplant / PAP

Zusatzumfang | bewusst optional

Kürzel	Vorgesehene Funktion	Priorität
V <Spalte>	Ghost-Vorschau ohne echten Zug	A · nach Kern
U	1-stufiges Undo im laufenden Spiel	A · nach Kern
H	Hilfe direkt im Spiel	A · nach Kern
I	Spielkennzahlen aus Board	B · Komfort
S	Direktes Speichern im Spiel	B · Komfort

Änderung am Grundmenü

Die Hauptmenüpunkte 0 bis 6 bleiben unverändert. Zusätzliche Befehle sind nur während der Partie erlaubt und verändern weder Siegbedingung noch Anzahl der Spieler.

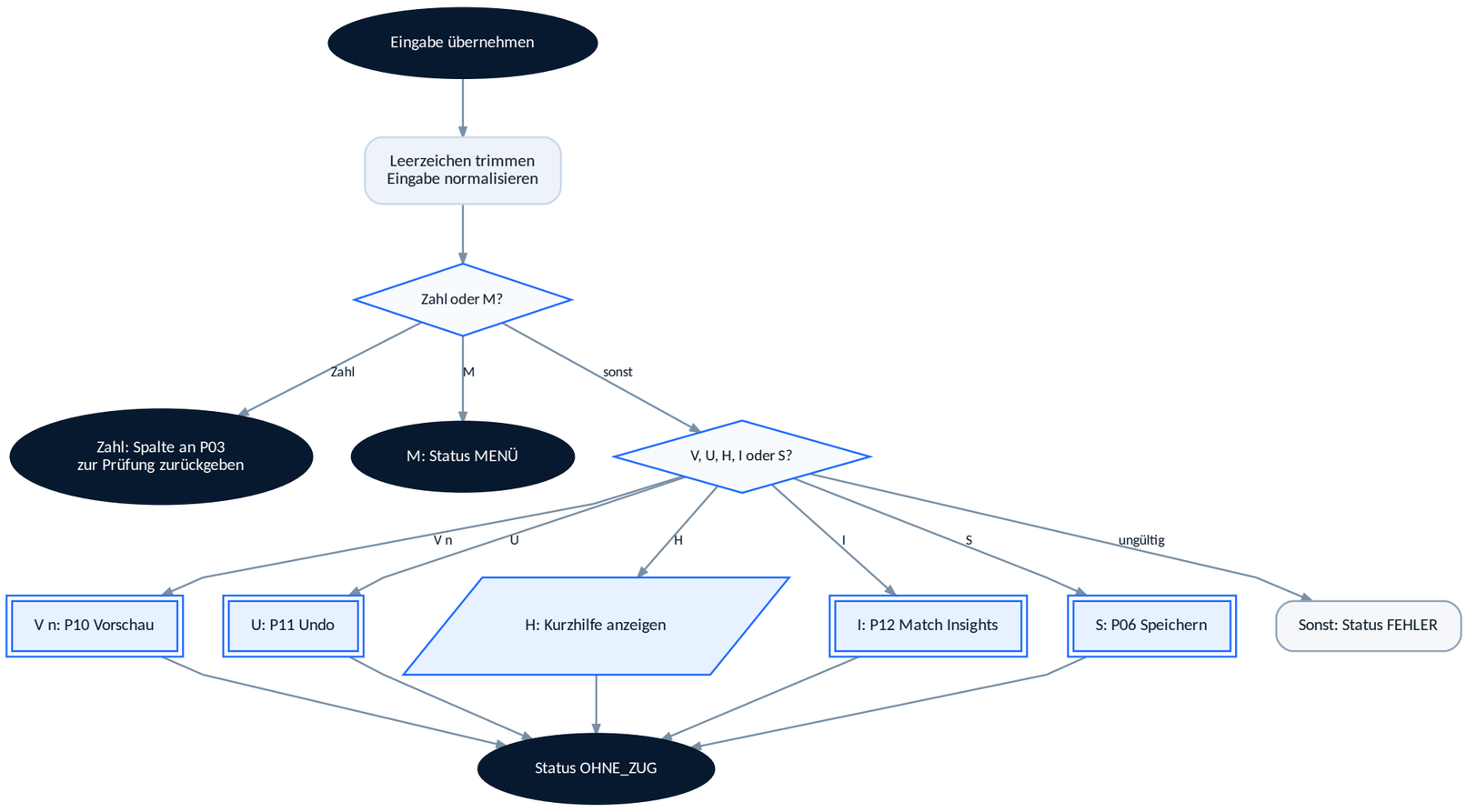
P09 / OPTIONALES KOMMANDOSYSTEM

Befehlssteuerung

Ein Dispatcher analysiert Spalteneingaben und Komfortbefehle.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / LESEN & ERKLÄREN

ZIEL

Parser und Spiellogik bleiben getrennt; neue Befehle lassen sich ergänzen.

EINGABE

Zahl, M, V n, U, H, I oder S

ERGEBNIS

Definierter Aktions- bzw. Rückgabestatus

SICHERHEIT

Unbekannte Befehle verursachen keine Zugänderung.

NACHWEIS

Jeder Sonderbefehl lässt aktuellen Spieler unverändert.

KURZABLAUF

01 Eingabe normalisieren

02 Zahl oder Kommandowort identifizieren

03 Befehl an passendes PAP weiterleiten

04 Status zur Zugsteuerung zurückgeben

© 2026 Julian Krauß | JK-PT-2026-051-PAP

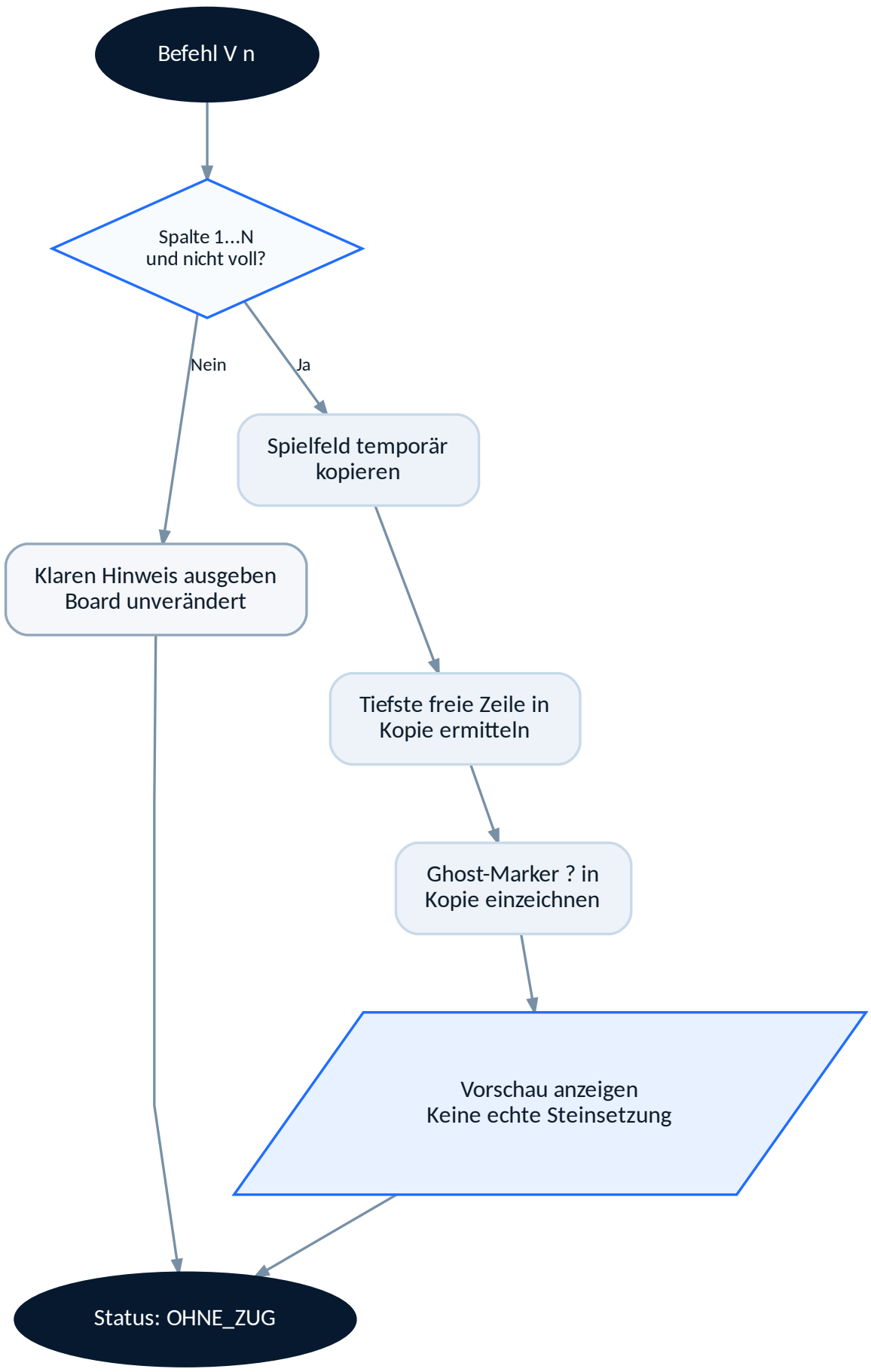
P10 / OPTIONALES FEATURE

Zugvorschau

Zeigt einen temporären Marker an der Landeposition.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / LESEN & ERKLÄREN

ZIEL

Eine Vorschau ist eine Sichtdarstellung und kein Spielzug.

EINGABE

Befehl V und Spaltennummer

ERGEBNIS

Markiertes Vorschaubrett

SICHERHEIT

Originalbrett bleibt vollständig unverändert.

NACHWEIS

Nach Vorschau kann dieselbe Person den echten Zug setzen.

KURZABLAUF

01 Spalte prüfen

02 Board kopieren

03 Freie Landereihe ermitteln

04 Vorschau ohne Steinsetzung ausgeben

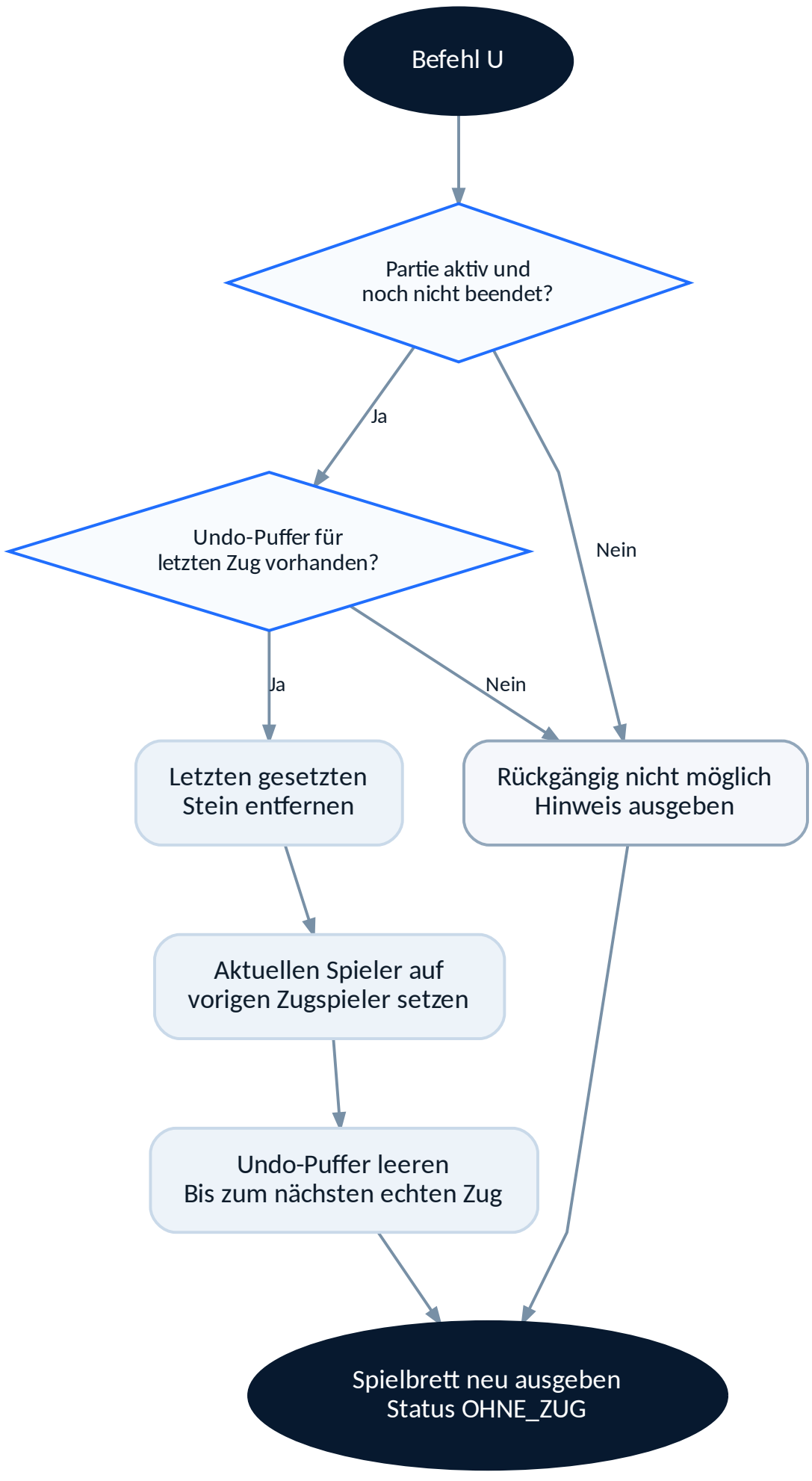
P11 / OPTIONALES FEATURE

Letzten Zug zurücknehmen

Ermöglicht genau einen Rücksprung im aktiven Spiel.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / LESEN & ERKLÄREN

ZIEL

Ein begrenzter Undo-Puffer ist klarer und leichter testbar als unbegrenzte Spielhistorie.

EINGABE

Befehl U

ERGEBNIS

Board und Spielerzustand zurückgesetzt

SICHERHEIT

Nach Spielende und nach Laden ist Undo deaktiviert.

NACHWEIS

Zweites U ohne neuen gültigen Zug wird abgewiesen.

KURZABLAUF

01 Spielstatus prüfen

02 Letzten gültigen Zug ermitteln

03 Letzten Stein entfernen

04 Vorigen Spieler wieder aktivieren

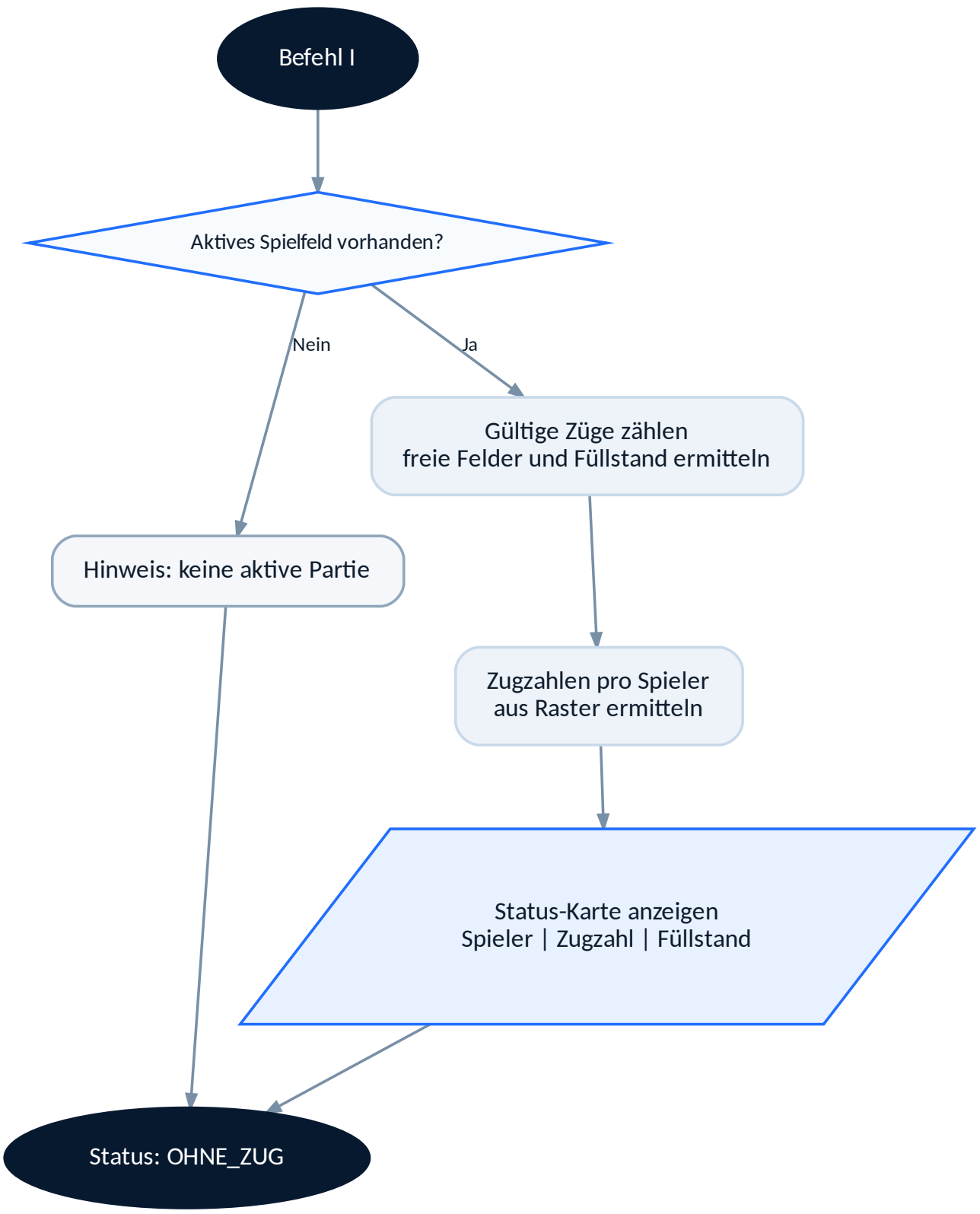
P12 / OPTIONALES FEATURE

Match Insights

Zeigt belegte Felder, freie Felder und Spielerstatistik.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / TECHNISCHE LESEHILFE

ZIEL

Die Kennzahlen stammen direkt aus dem Brett statt aus fehleranfälligen Parallelzählern.

KONTROLLPUNKT

Ohne laufende Partie nur einen Hinweis ausgeben.

ABNAHMETEST

Die angezeigten Summen entsprechen exakt dem Board.

15 / TECHNISCHES KONZEPT

Objektorientierte Architektur

Klassenverantwortlichkeiten und Erweiterungspunkte eindeutig zuordnen.

Klasse	Verantwortung
Application	Menü 0–6, Lebenszyklus, Weiterleiten
Game	Spielzustand, aktiver Spieler, Spielphase, Zugwechsel
Board	Dynamisches Raster, Einwurf, Undo-Entfernen, Gewinnprüfung
Player	Name, Symbol, Siege (Anzeige aus Rangliste)
CommandParser	Text zu Zahl, V, U, H, I, S, M interpretieren
MoveHistory	Letzten gültigen Zug vorhalten, Undo-Verfügbarkeit
SaveManager	Prüfen, Schreiben und Laden in temporärer Struktur
HighscoreManager	Siege persistent führen, sortieren, anzeigen
ConsoleUI	Eingabe, Darstellung, Hilfe, Vorschau, Statistik

Kapselung als Grundregel

Game steuert, Board kennt Rasterregeln und ConsoleUI gibt aus. Dateiformate bleiben im SaveManager. Dadurch wird jede Klasse für sich testbar; doppelte Zustandsverwaltung wird vermieden.

16 / BEDIENUNG

Interface | Terminal-Konzept

Gezielte Modernisierung der Konsole, ohne unnötige Bibliotheken oder Internetzugriff.

VIER GEWINNT / KONZEPTANSICHT

MODUS: 3 SPIELER FELD: 9 x 8 ZUG: 12

```
  1 2 3 4 5 6 7 8 9

. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. X . O . . . . .
X O X O . ? . . .
```

ANNA (X) | BEN (O) | MIA (#)

Aktion: 1-9 | V 6 | U | H | I | S | M

V 6: Vorschau, noch kein echter Zug.

Legende: X/O/# sind nur Beispielsymbole, „?“ ist der temporäre Vorschau-Marker, „.“ markiert freie Zellen. Vorschau ist im echten Board nicht gespeichert. In der fertigen Konsole sind monochrome Symbole Pflicht; optionale ANSI-Farben dürfen die Lesbarkeit nur ergänzen.

Designprinzipien

- Klare Rasterbreite, Spaltennummern über jedem Feld und sichtbarer aktueller Spieler.
- Konsistente Fehlertexte statt Programmabbruch oder unlesbarer Konsolenmeldungen.
- Einfaches Standard-Terminal als Pflicht; Farbdarstellung nur als progressive Verbesserung.

Wirkung von Sonderbefehlen

Wichtigster Nachweis: Nur eine gültige Spalteneingabe darf einen Spielzug erzeugen.

Eingabe	Raster geändert?	Spielerwechsel?	Highscore?
5	Ja, falls frei	Ja, ohne Gewinn	Nur bei Gewinn
V 5	Nein	Nein	Nein
U	Ja, nur letztes Feld	Zum vorigen Spieler	Nein
H	Nein	Nein	Nein
I	Nein	Nein	Nein
S	Nein	Nein	Nein
M	Nein	Nein	Nein
Ungültig	Nein	Nein	Nein

Explizite Sonderregeln

- Vorschau: arbeitet auf einer Kopie; bei voller oder ungültiger Spalte erscheint ein Fehlerhinweis.
- Undo: erlaubt nur den letzten Zug innerhalb der aktuellen Spielsitzung und nie nach Sieg oder Remis.
- Laden: setzt den Undo-Puffer zurück. Ein vor dem Laden entstandener Zug ist nicht rücknehmbar.
- Speichern: dokumentiert nachweisbar Spielstand, nicht die laufende Undo-Historie.
- Sieg: wird exakt einmal gespeichert; am Spielende sind weitere Aktionen deaktiviert.

18 / ALGORITHMIK

Exakte Gewinnprüfung

Erklärung für die mündliche Vorführung, inklusive Grenzprüfungen.

RICHTUNGEN AB LETZTEM STEIN

(1,0) (0,1) (1,1) (1,-1)

vertikal | horizontal | diagonal / diagonal

Logik je Richtungsvektor

- Starte beim zuletzt gesetzten Spielstein mit Koordinate (reihe, spalte).
- Zähle gleiche Symbole in Richtung (dr, dc), bis Rand oder Fremdsymbol erreicht ist.
- Zähle dasselbe in Gegenrichtung (-dr, -dc).
- 1 + vorwärts + rückwärts ≥ 4 bedeutet Gewinn.
- Erst danach prüfen, ob das Feld voll ist. Nur andernfalls wechselt der Spieler.

Warum dieser Algorithmus?

Anstatt alle Feldpositionen neu zu prüfen, beginnt die Kontrolle nur beim zuletzt gesetzten Stein. Jede der vier Richtungen wird vorwärts und rückwärts verfolgt. Das vermeidet unnötige Arbeit und vereinfacht die Erklärung am Code. Grenzkoordinaten werden vor einem Rasterzugriff geprüft.

19 / TESTPROTOKOLL

Extras beweisbar testen

Soll-Verhalten ist definiert. Ist-Ergebnisse müssen nach der Implementierung eingetragen werden.

ID	Testaktion	Erwartung
E01	Vorschau freie Spalte	? nur in Kopie; original unverändert
E02	Vorschau volle Spalte	Fehlermeldung; Spieler bleibt
E03	Undo direkt nach Zug	Stein weg; richtiger Spieler aktiv
E04	Undo zweimal	Zweiter Undo verweigert
E05	Undo nach Laden	Nicht verfügbar; Brett bleibt korrekt
E06	Hilfe H	Kommandos lesbar, kein Zug verbraucht
E07	Insights I	Züge / freie Felder / Prozent korrekt
E08	Quick Save S	Datei korrekt; aktueller Spieler gleich
E09	Ungültiger Befehl	Hinweis, gleiche Person erneut
E10	Gewinn → Highscore	Sieg exakt +1, keine spätere Undo

Nachweisfelder im echten Projekt: Testdatum, Programmversion/Commit, Ist-Ausgabe, Ergebnis (bestanden/nicht bestanden) und gegebenenfalls Screenshot. Diese Entwurfsfassung enthält bewusst keine erfundenen Testergebnisse.

20 / DATENQUALITÄT

Speichern, Laden, Undo sicher verbinden

Die Erweiterungen dürfen keinerlei bestehende Kernfunktion beschädigen.

Drei Zustandsbereiche

Dauerhafter Spielstand

Version, Anzahl und Namen der Spieler, Symbole, aktueller Spieler, Rastergröße, Rasterinhalt und Spielphase. Ein Lesefehler darf den gültigen Zustand nicht überschreiben.

Flüchtiger Undo-Puffer

Speichert ausschließlich die letzte gültige Zugaktion mit Zeile, Spalte und Spielerindex. Bei erfolgreichem Laden oder Spielende wird der Puffer zurückgesetzt.

Persistente Highscores

Unabhängige Rangliste. Bei Gewinn genau ein Update; nach Fehler keine falsche Erfolgsmeldung. Ein Rückgängig nach Partiende ist nicht zulässig.

Schadensfreie Dateiverarbeitung

- In eine temporäre Datei schreiben und Erfolg prüfen, bevor die echte Datei ersetzt wird.
- Beim Laden zuerst eine temporäre Game-Struktur aufbauen; erst nach vollständiger Validierung übernehmen.
- Spielstandformat mit Versionskennung versehen, unbekannte Formate sicher ablehnen.
- Spaltenzahl, Reihenanzahl, Symbole, Spielerzahl und aktiven Spielerindex zusammen prüfen.

21 / SCHULPRÄSENTATION

Live-Demo mit Beweisführung

Ein vorbereitetes Drehbuch reduziert Rückfragen und macht die Arbeitsweise nachvollziehbar.

01 **Menü**
Sechs Funktionen + Beenden zeigen; ungültige Auswahl testen.

02 **Zwei Spieler**
Spieler erfassen; 7×6 Feld und Zugwechsel nachweisen.

03 **Vorschau V 4**
Ghost-Stein zeigen und verdeutlichen: kein echter Zug.

04 **Undo U**
Gültigen Zug zurücknehmen; Spieler korrekt wieder aktiv.

05 **Hilfe + Insights**
H / I, inkl. Zugzahl und Füllstand erläutern.

06 **Schnellspeichern**
S betätigen; laden und identisches Raster zeigen.

07 **Vier Spieler**
11×10 Feld; modulo 4 und Eingabefehler demonstrieren.

08 **Gewinn + Highscore**
Vorbereiteten Spielstand gewinnen; +1 in Rangliste zeigen.

09 **Code + GitHub**
Board, Game, P03/P09/P11, Commits und Tests erläutern.

Präsentationsziel: 27 Minuten Inhalt plus rund 3 Minuten Puffer/Fragen gemäß Aufgabenblatt. Neue Funktionen sind nur Teil der Demo, wenn vollständig umgesetzt und getestet.

22 / FACHGESPRÄCH

Antworten auf typische Lehrerfragen

Keine auswendig gelernten Floskeln: kurze, überprüfbare technische Begründungen.

01 Warum genau diese Klassen?

Klassen teilen Zuständigkeiten auf: Board kennt Rasterregeln, Game koordiniert den Ablauf, UI spricht mit dem Benutzer und Manager speichern Dateien.

02 Warum wächst das Spielfeld?

Das Aufgabenblatt legt 7×6 als Grundfläche fest. Pro weiterem Spieler addieren wir zwei Zeilen und zwei Spalten. Die Vierer-Gewinnregel bleibt konstant.

03 Warum wird Undo beim Laden zurückgesetzt?

Das Undo ist bewusst nur ein temporärer Ein-Schritt-Puffer. Ohne gespeicherte Zug-Historie wäre eine sichere Rücknahme nach Laden nicht gewährleistet.

04 Warum ändert Vorschau nichts?

Die Zugvorschau arbeitet auf einer temporären Rasterkopie. Dadurch bleibt der echte Zustand unverändert und es findet kein Spielerwechsel statt.

05 Wie vermeidet ihr doppelte Siege?

Nach einem Gewinn wechselt die Spielphase auf beendet; die Highscore-Aktualisierung erfolgt genau einmal während dieses Übergangs.

06 Wie schützt ihr das Laden?

Die Datei wird vollständig in einen Prüfzustand gelesen. Erst wenn Version, Werte und Dimensionen gültig sind, wird der Zustand in die laufende Partie übernommen.

23 / CODEKONZEPT

Methoden & Pseudocode

Lernhilfe zum Verständnis: kein Nachweis einer fertigen Anwendung.

Zug setzen und prüfen | Ablauf-Skizze

```
if (!board.isValidColumn(column)) return ERROR;
if (board.isColumnFull(column))  return ERROR;
auto move = board.place(column, current.symbol);
undoBuffer = move;
if (board.checkWinFrom(move)) {
    game.finish(WIN);
    highscore.addWinOnce(current.name);
} else if (board.isFull()) {
    game.finish(DRAW);
} else {
    currentIndex = (currentIndex + 1) % playerCount;
}
```

Wichtig: Dies ist Pseudocode; Funktionsnamen sind nur Vorschläge. Der C++-Quelltext muss mit einem vollständigen Klassendesign, überprüften Fehlerpfaden, geeigneten Rückgabewerten und Tests umgesetzt werden.

Wesentliche API-Ideen

- Board::placeToken(column, symbol) gibt Position und Erfolg/Fehler zurück.
- Board::checkWinFrom(row, col, symbol) prüft vier Richtungen vom gesetzten Stein.
- Game::undoLastMove() prüft die Spielphase und stellt den vorherigen Zustand wieder her.
- CommandParser::parse(line) liefert einen klaren Aktionstyp statt verschachtelter Texteingaben.

24 / PROJEKTFÜHRUNG

Umsetzung nach Priorität

Zusätzliche Funktionen bleiben ein Bonus, die Pflichtfunktionen sind die Bewertungsgrundlage.

Empfohlene Entwicklungsreihenfolge

01 **Kernspiel**
Main-Menü, Board, Player, echte Spielzüge, Fehlerprüfung.

02 **Regelwerk**
Gewinn horizontal/vertikal/diagonal und Remis mit Tests.

03 **Daten & OOP**
Speichern/Laden, Highscore, saubere Klassenstruktur, GitHub.

04 **Extras A**
Hilfe H, Ghost-Vorschau V, Undo U.

05 **Extras B**
Live-Insights I, Schnellspeichern S.

06 **Abnahme**
Tests, Handout, Sourcecode-Verständnis, Demo, KI-Prompts.

Risiken und Gegenmaßnahmen

- Zeit: Zusätze dürfen die festen Meilensteine aus Oktober bis Februar nicht verschieben.
- Komplexität: Jede neue Funktion braucht einen eigenen, reproduzierbaren Testfall.
- Präsentation: Nicht alles vorführen, sondern Kernfunktion, einen Fehlerfall und zwei Extras gezielt zeigen.

25 / REFERENZEN

Design, Quellen & Status

Klare Trennung zwischen Aufgabe, eigener Gestaltung und unterstützender KI-Arbeit.

Dokumentierte Grundlagen

Aufgabenblatt 51 | Dr. Michael Litera

Grundregeln, Spielerzahl und Feldskalierung (Seiten 2–3), Persistenz und Highscore (S. 4), Termine (S. 6–7), Tests und Bewertungsmatrix (S. 8, 12–13).

Corporate Design Guide | Julian Krauß

Design-ID JK-CD-DE-2026-003, Revision 3.2, Release 4.6.3. Navy, Systemblau, Cyan, Segoe-/Aptos-orientierte Systemtypografie, nummerierte Module und klare Werkkennzeichnung.

KI-Unterstützung | transparent

KI wurde zur Strukturierung, Diagrammerstellung, Ideenentwicklung und sprachlichen Dokumentation eingesetzt. Fachliche Prüfung, endgültiger C++-Code und tatsächliche Testergebnisse müssen eigenständig nachvollzogen werden.

Version / Nutzungsstatus

Feld	Angabe
Werk-ID	JK-PT-2026-051-PAP
Design-ID	JK-CD-DE-2026-003
Ausgabe	Edition 2.0 | 22. Oktober 2026
Status	Planungsentwurf; Umsetzung und Tests ausstehend
Zusatzfunktionen	Projekterweiterungen, nicht Teil der Pflichtvorlage

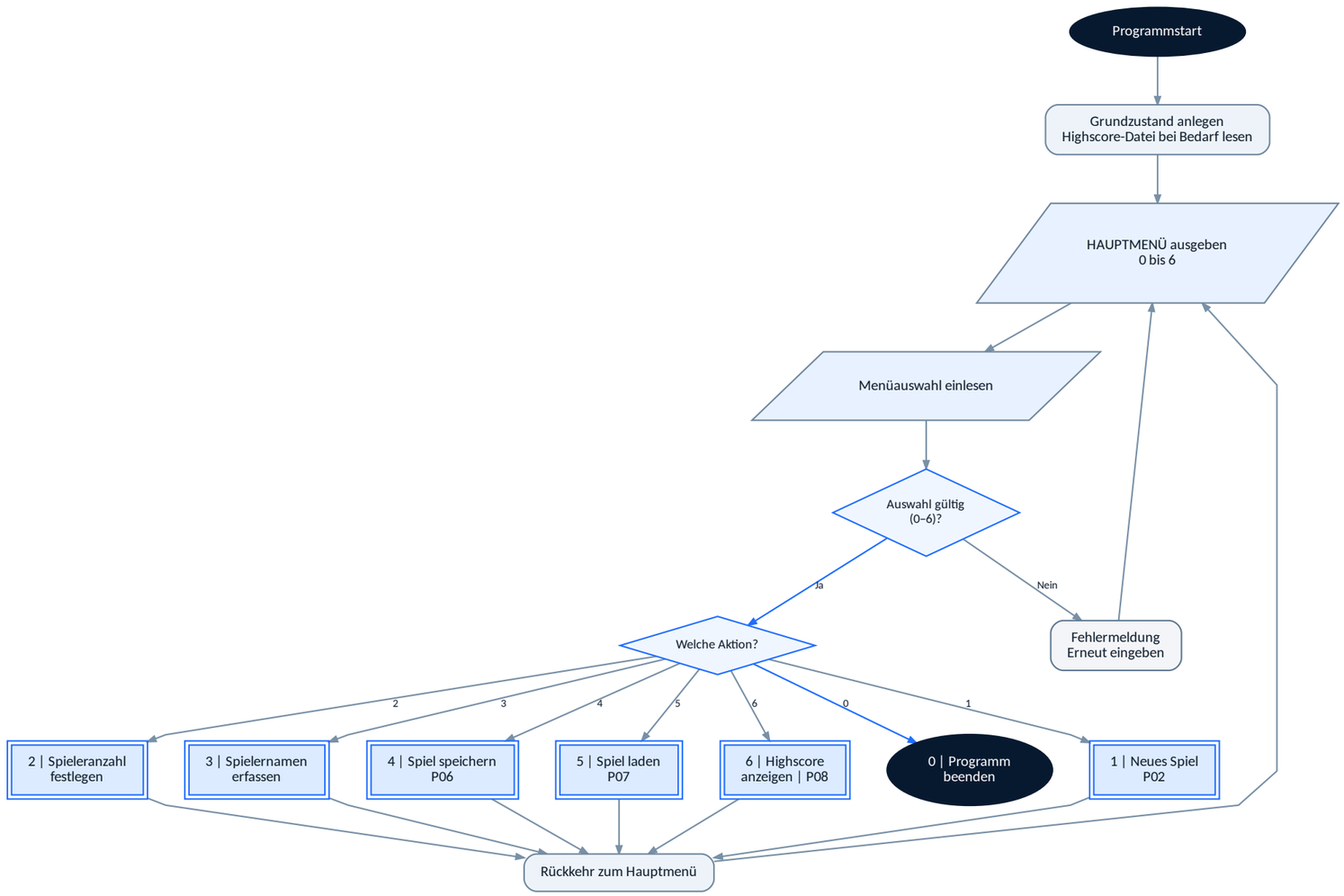
P01 / PROGRAMMFÜHRUNG

Hauptmenü

Die zentrale Navigation mit gültiger Auswahl, Fehlerpfad und Rücksprung.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / LESEN & ERKLÄREN

ZIEL

Das Menü ist die einzige Eintrittsstelle für Spielstart, Konfiguration und persistente Daten.

EINGABE

Menüeingabe 0 bis 6

ERGEBNIS

Aufruf einer Funktion oder sauberes Programmende

SICHERHEIT

Unbekannte Eingabe wird erneut abgefragt.

NACHWEIS

Ungültige Auswahl 7 bzw. Buchstaben führt nicht zum Absturz.

KURZABLAUF

01 Menü mit sechs Aktionen ausgeben

02 Auswahl einlesen und validieren

03 Gewählten Teilablauf starten

04 Zum Hauptmenü zurückkehren

© 2026 Julian Krauß | JK-PT-2026-051-PAP

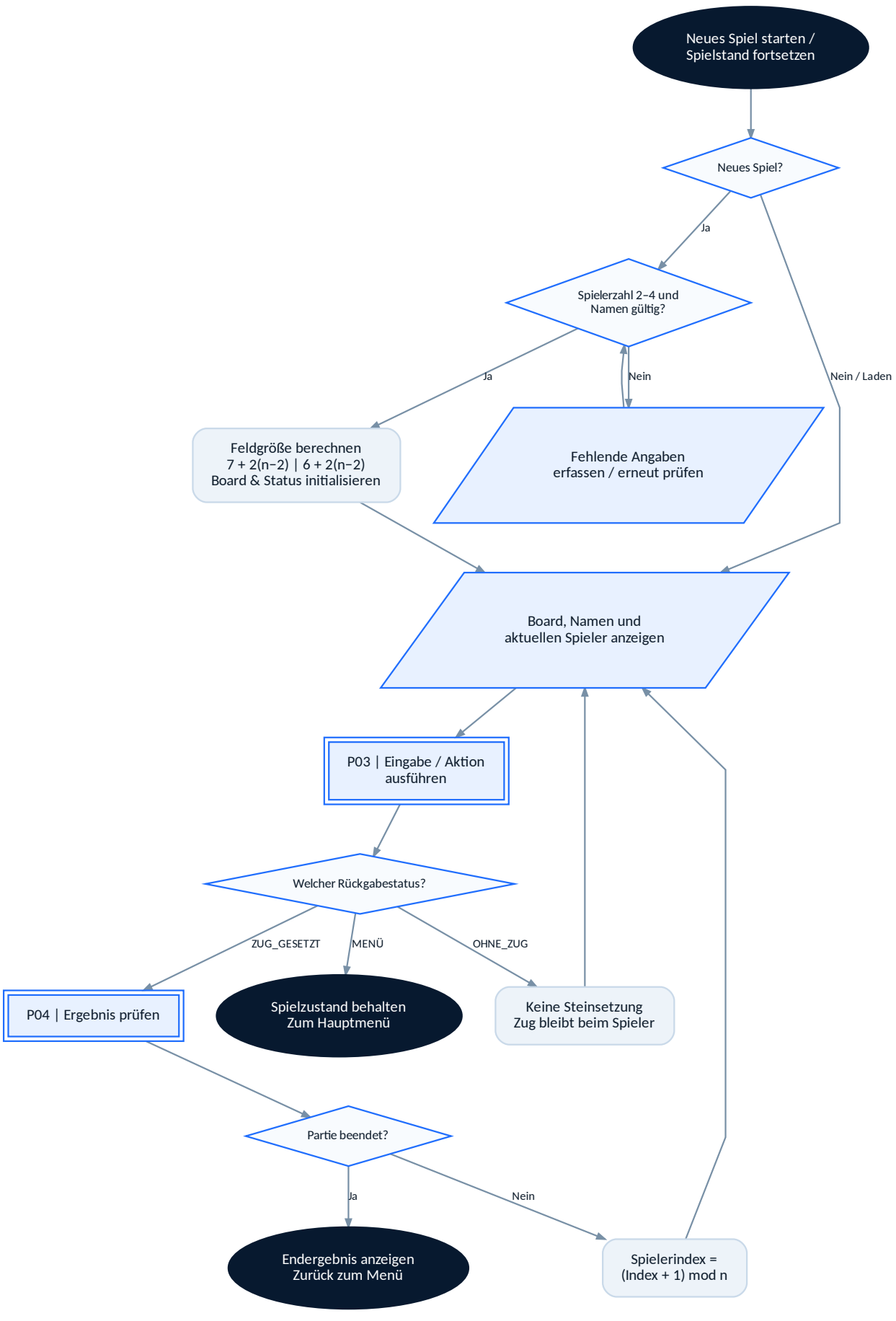
P02 / PARTIEVERWALTUNG

Spielsteuerung

Steuert die Spielphase vom Start oder Laden bis zum Ergebnis.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / LESEN & ERKLÄREN

ZIEL

Trennung von Eingabe, Brettregeln und Partiezustand verhindert doppelte Geschäftslogik.

EINGABE

Konfiguration oder geladener Spielstand

ERGEBNIS

Neuer Zug, Pause oder abgeschlossenes Spiel

SICHERHEIT

Zusatzbefehle verbrauchen keinen Spielzug.

NACHWEIS

Spielerwechsel nur nach gültigem Steinsetzen.

KURZABLAUF

01 Spielerangaben validieren

02 Board passend dimensionieren

03 Spielaktion an P03 weitergeben

04 Ergebnis P04 abfragen und Zustände wechseln

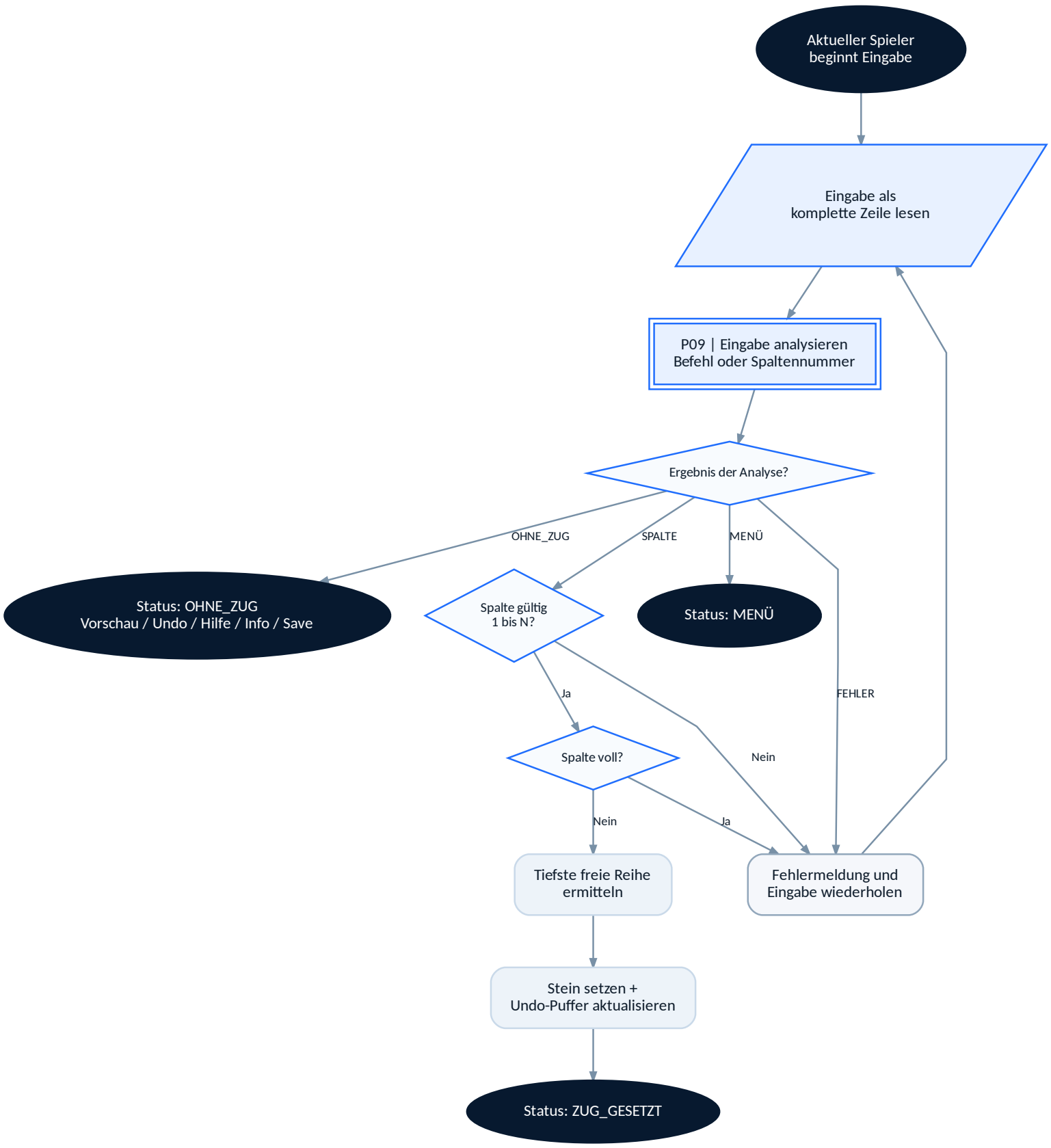
P03 / EINGABELOGIK

Spielzug und Validierung

Unterscheidet Menübefehle und gültige Zug-Eingaben.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / TECHNISCHE LESEHILFE

ZIEL

Eine einzige Eingaberoutine entscheidet über gültige Spalten und verhindert Zustandsfehler.

KONTROLLPUNKT

Volle Spalten, falsche Zahlen und ungültige Texte werden abgefangen.

ABNAHMETEST

Vorschau, Undo und Hilfe dürfen den Zug nicht verbrauchen.

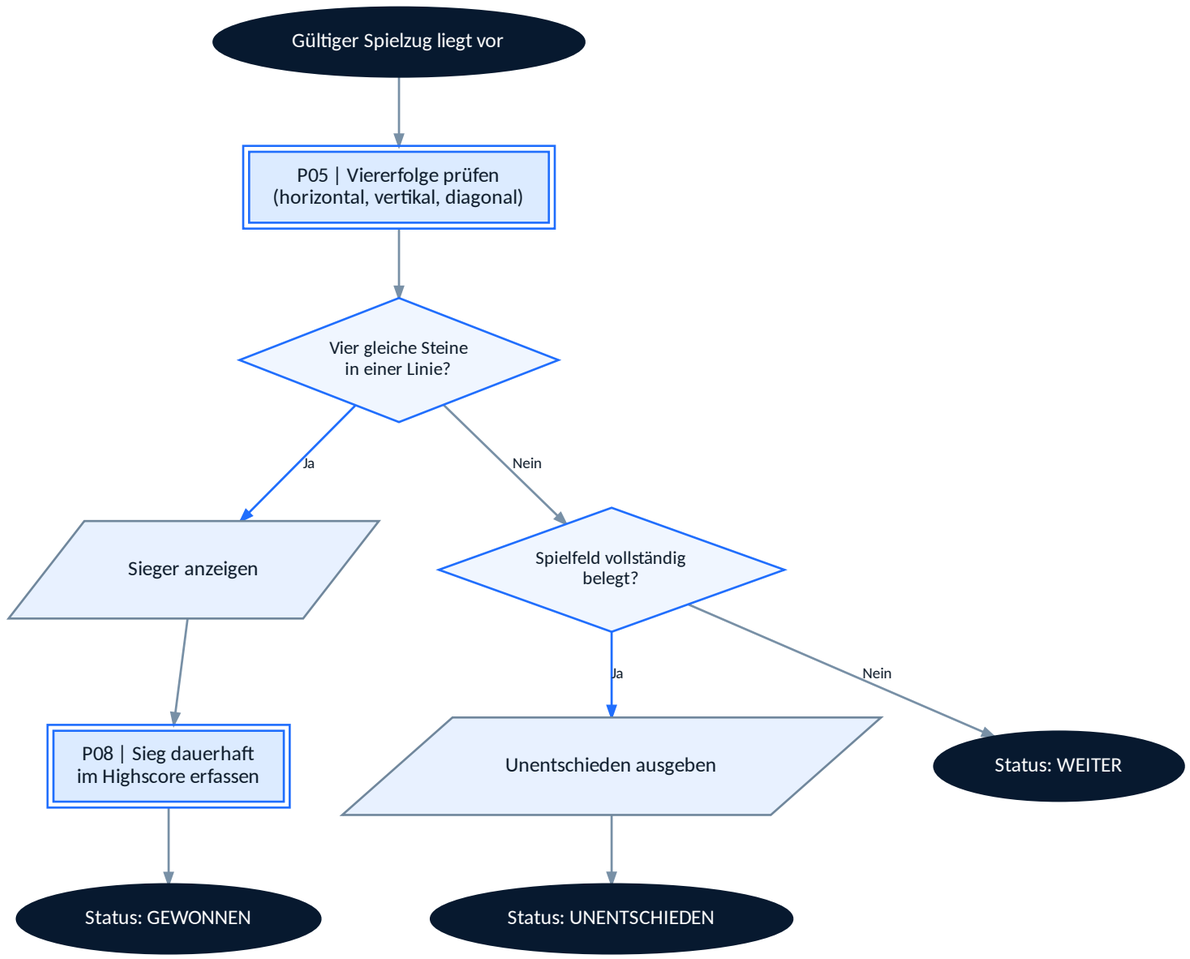
P04 / ENDZUSTAND

Spielergebnis

Unterscheidet Sieg, Unentschieden und Fortsetzung der Partie.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / LESEN & ERKLÄREN

ZIEL

Die Reihenfolge ist wichtig: Erst Gewinnprüfung, dann Remis und zuletzt Spielerwechsel.

EINGABE

Zuletzt gesetzter Stein

ERGEBNIS

Gewonnen, unentschieden oder weiterspielen

SICHERHEIT

Highscore höchstens einmal pro Gewinn aktualisieren.

NACHWEIS

Volles Feld mit Viererreihe bleibt ein Sieg, kein Remis.

KURZABLAUF

01 Gewinnprüfung aufrufen

02 Bei Sieg Ergebnis sichern

03 Nur danach volles Brett prüfen

04 Anderenfalls Partieverlauf fortsetzen

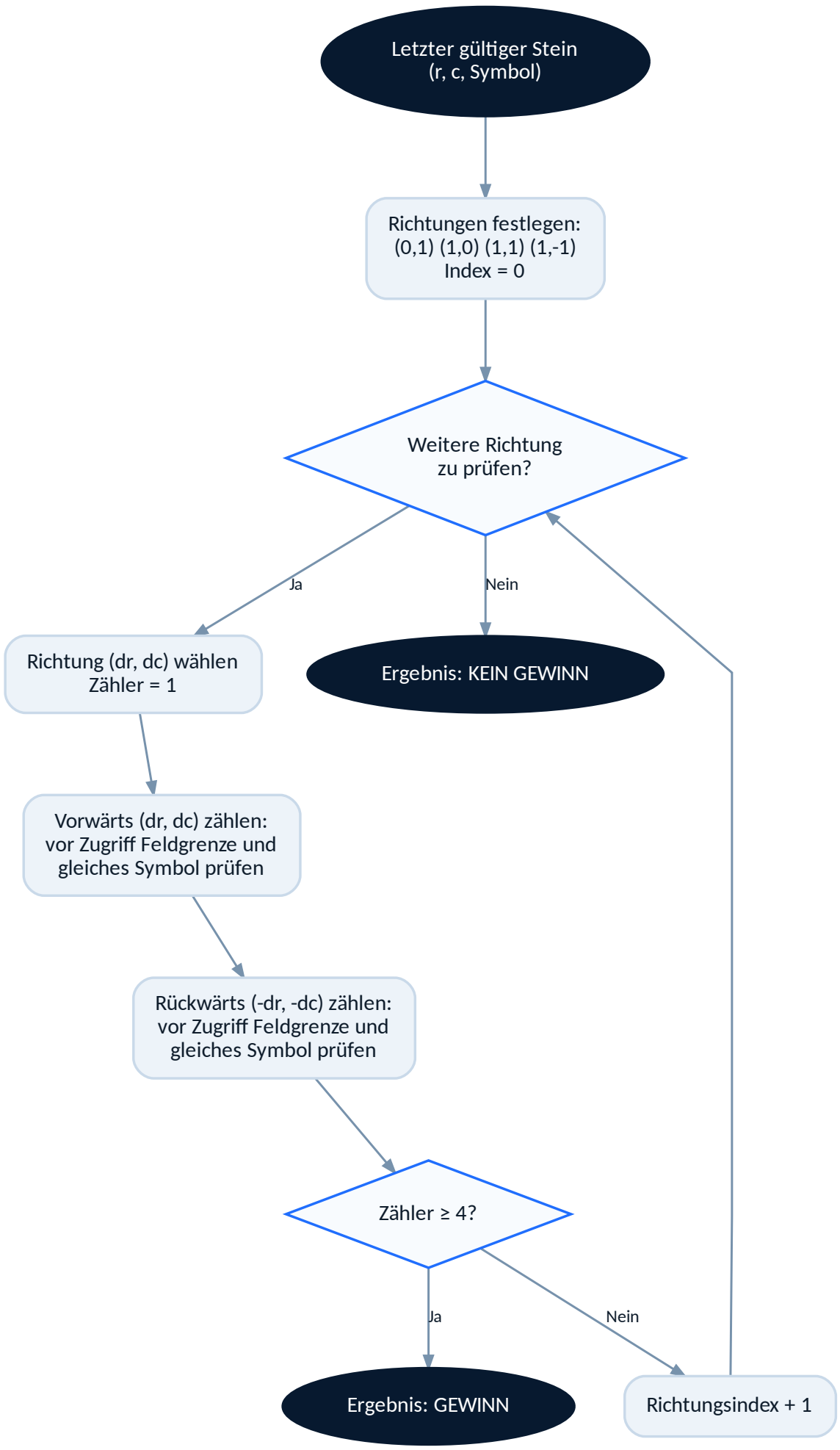
P05 / SPIELREGEL

Gewinnprüfung

Prüft vier verbundene Symbole horizontal, vertikal und diagonal.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / LESEN & ERKLÄREN

ZIEL

Die vier Gewinnrichtungen gelten unabhängig von der Spielfeldgröße.

EINGABE

Board, Symbol und letzter Zug

ERGEBNIS

true / false

SICHERHEIT

Rastergrenzen vor jedem Zellenzugriff prüfen.

NACHWEIS

Alle vier Richtungen und diagonale Randfälle abdecken.

KURZABLAUF

01 Jede Gewinnrichtung prüfen

02 Gleiche Symbole zusammenzählen

03 Rand oder Fremdsymbol stoppt

04 Ab vier verbundenen Steinen: Gewinn

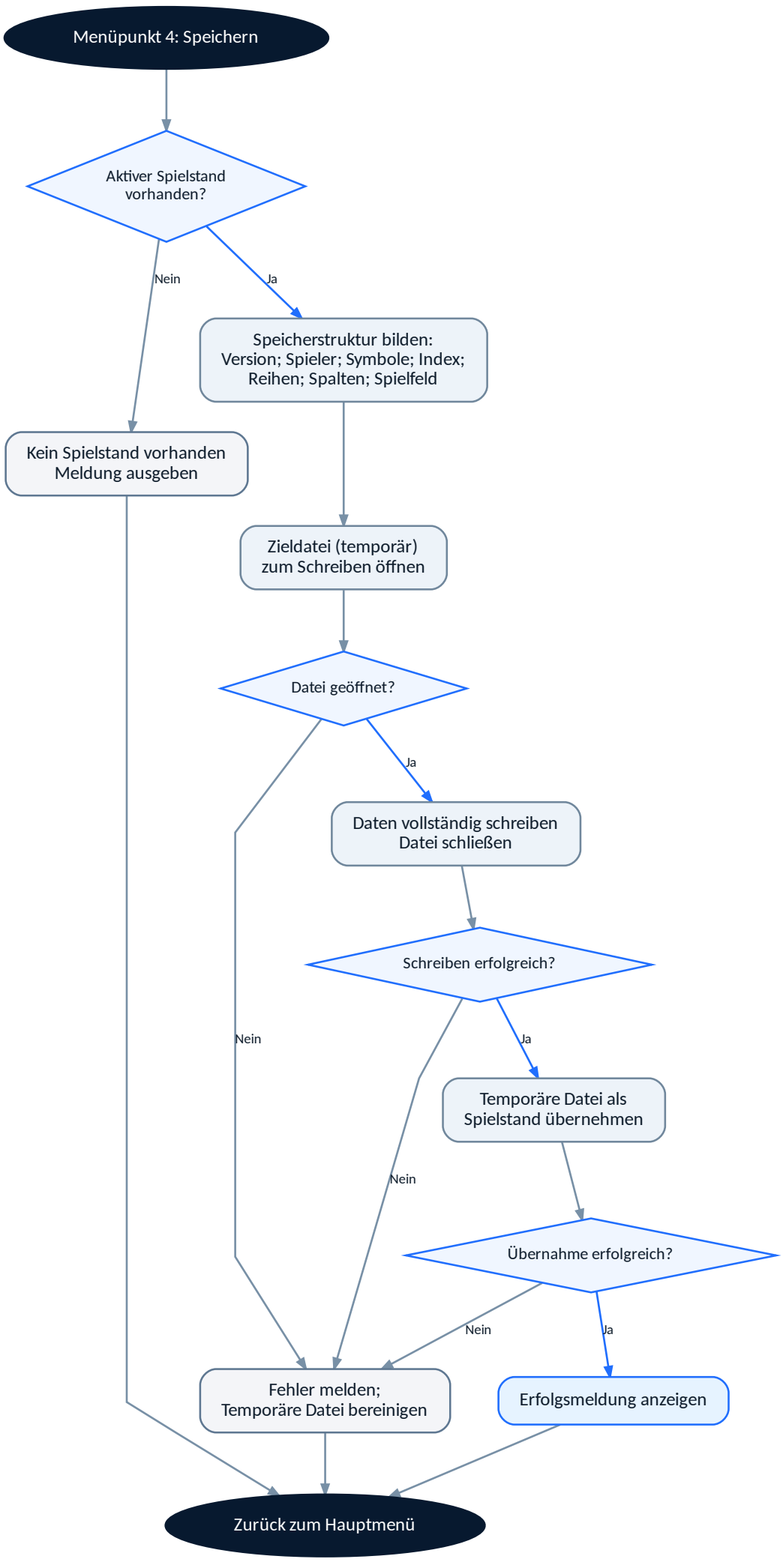
P06 / DATEIOPERATION

Speichern

Schreibt einen reproduzierbaren, vollständigen Spielzustand.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / LESEN & ERKLÄREN

ZIEL

Ein Spielstand muss nach Neustart dieselben Spieler, Dimensionen und aktiven Spieler erzeugen.

EINGABE

Gültiger Spielzustand

ERGEBNIS

Gespeicherte Datei oder Fehlermeldung

SICHERHEIT

Schreibfehler als Fehler behandeln und nicht als Erfolg melden.

NACHWEIS

Datei nach Neustart prüfen und wieder öffnen.

KURZABLAUF

01 Spieldaten erfassen

02 Datenformat samt Version erzeugen

03 Datei schreiben und prüfen

04 Ergebnis verständlich melden

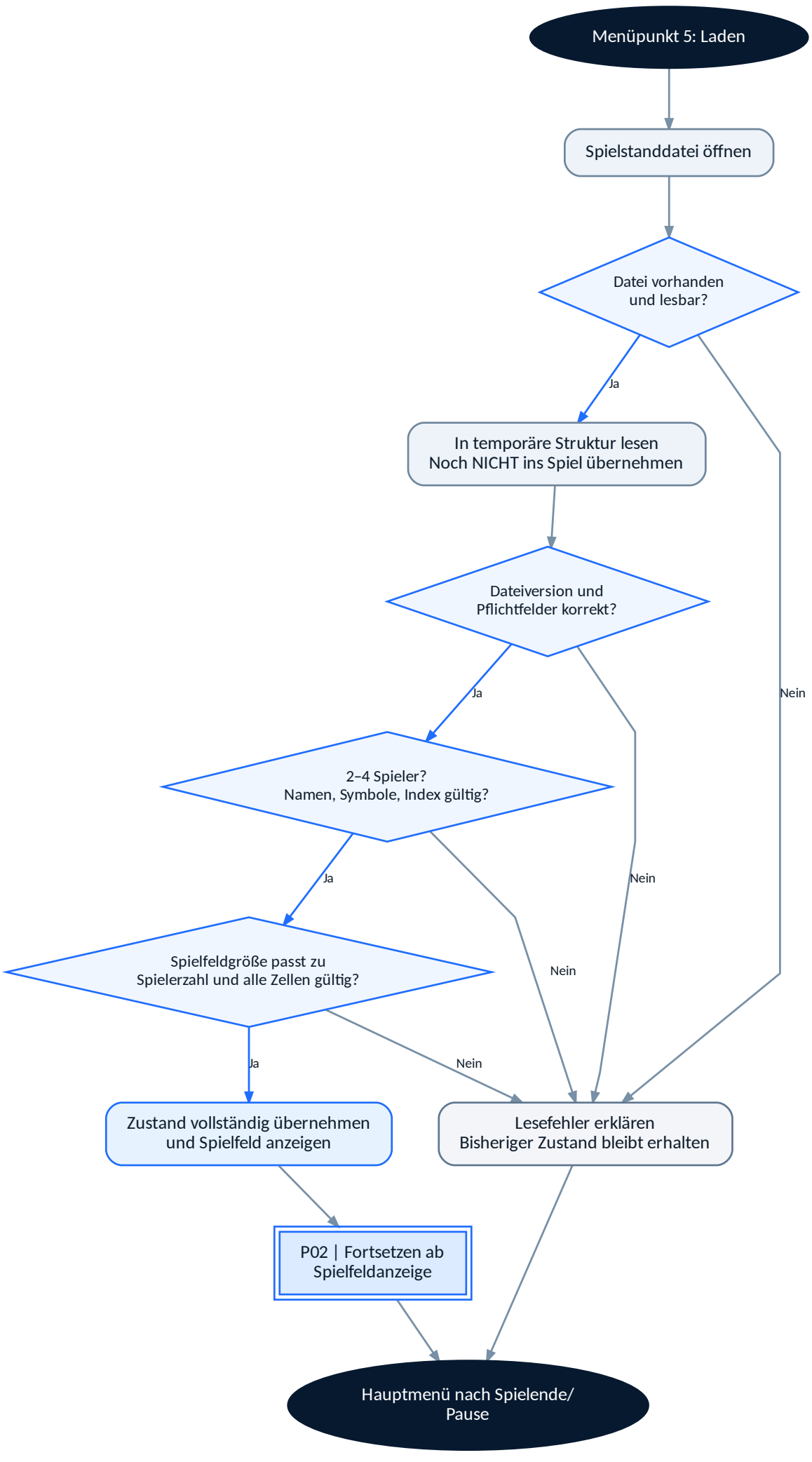
P07 / DATEISICHERHEIT

Laden und Prüfen

Übernimmt eine Datei erst nach vollständiger Validierung.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / LESEN & ERKLÄREN

ZIEL

Unvollständige oder beschädigte Dateien dürfen die laufende Partie nicht zerstören.

EINGABE

Dateipfad und Dateiinhalt

ERGEBNIS

Geprüfter Spielstand oder Fehlertext

SICHERHEIT

Erst in temporäre Struktur lesen, danach übernehmen.

NACHWEIS

Fehlende Datei, falsche Dimensionen, defektes Raster.

KURZABLAUF

01 Datei öffnen und Version prüfen

02 Alle Felder validieren

03 Spielzustand konsistent herstellen

04 Erst dann aktives Spiel ersetzen

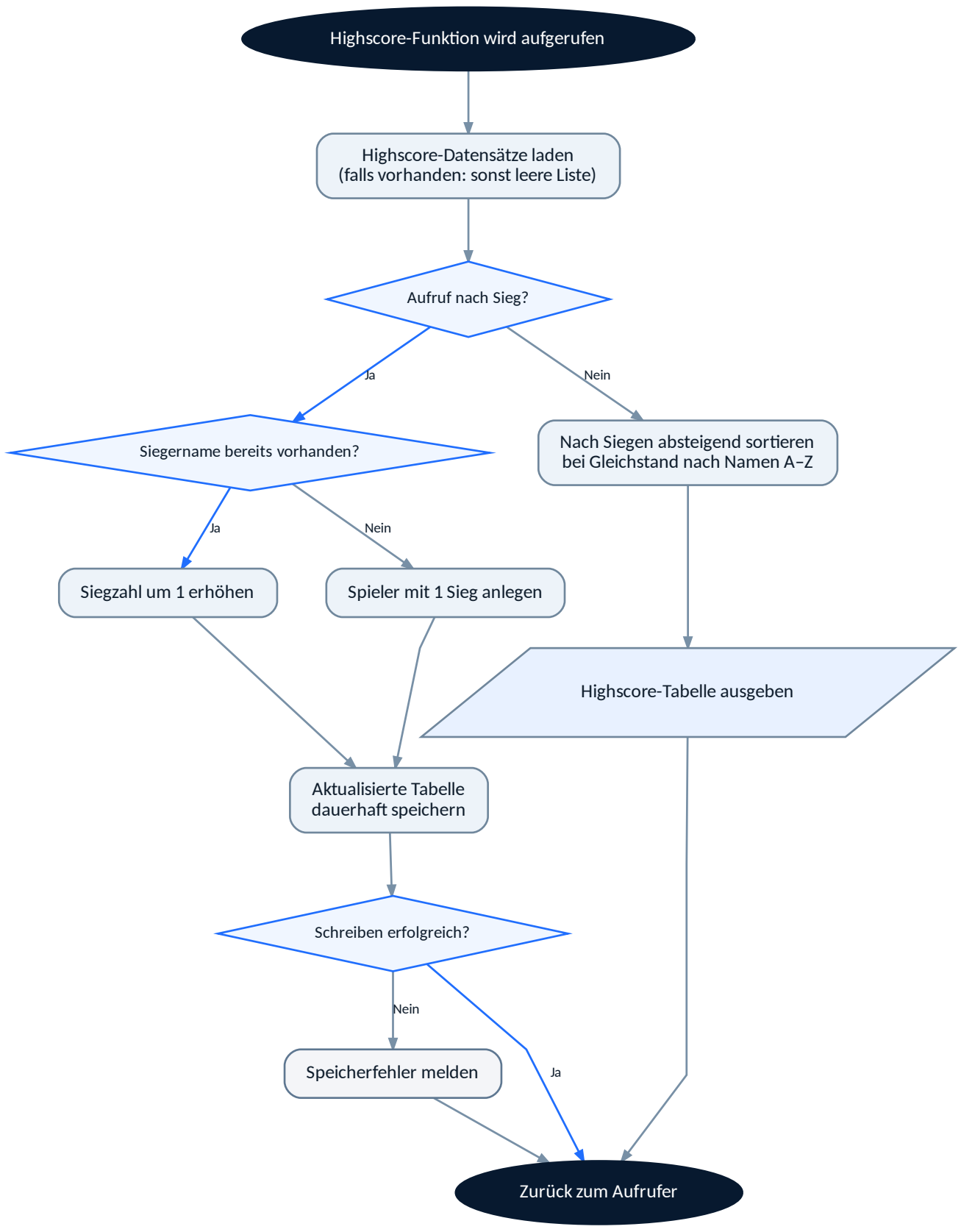
P08 / RANGLISTE

Highscore

Verwaltet Siege dauerhaft und sortiert absteigend.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / TECHNISCHE LESEHILFE

ZIEL

Die Rangliste ist vom einzelnen Spielstand getrennt.

KONTROLLPUNKT

Gleichstand nachvollziehbar alphabetisch ordnen.

ABNAHMETEST

Ein Sieg erhöht genau einen Datensatz um eins.

© 2026 Julian Krauß | JK-PT-2026-051-PAP

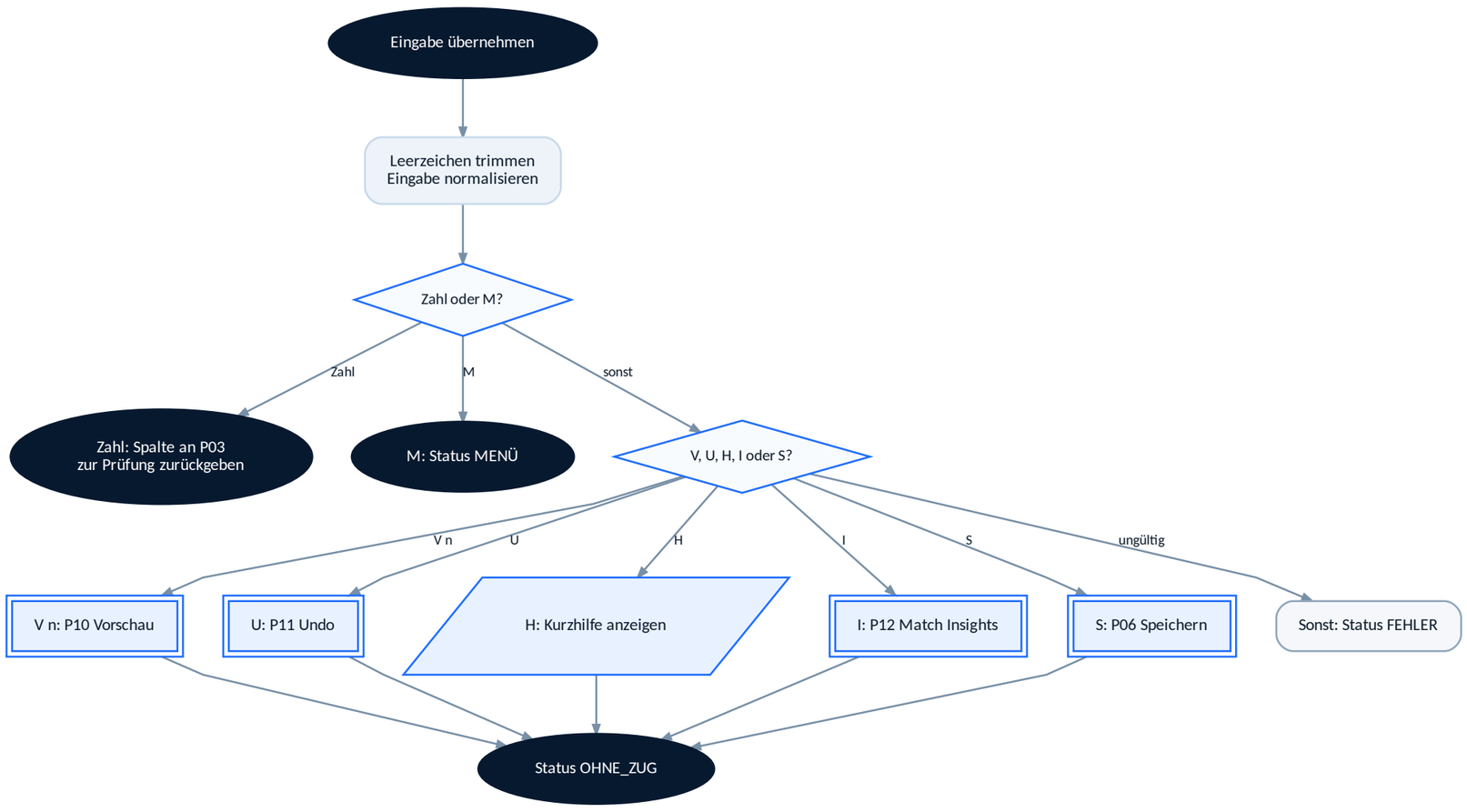
P09 / OPTIONALES KOMMANDOSYSTEM

Befehlssteuerung

Ein Dispatcher analysiert Spalteneingaben und Komfortbefehle.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / LESEN & ERKLÄREN

ZIEL

Parser und Spiellogik bleiben getrennt; neue Befehle lassen sich ergänzen.

EINGABE

Zahl, M, V n, U, H, I oder S

ERGEBNIS

Definierter Aktions- bzw. Rückgabestatus

SICHERHEIT

Unbekannte Befehle verursachen keine Zugänderung.

NACHWEIS

Jeder Sonderbefehl lässt aktuellen Spieler unverändert.

KURZABLAUF

01 Eingabe normalisieren

02 Zahl oder Kommandowort identifizieren

03 Befehl an passendes PAP weiterleiten

04 Status zur Zugsteuerung zurückgeben

© 2026 Julian Krauß | JK-PT-2026-051-PAP

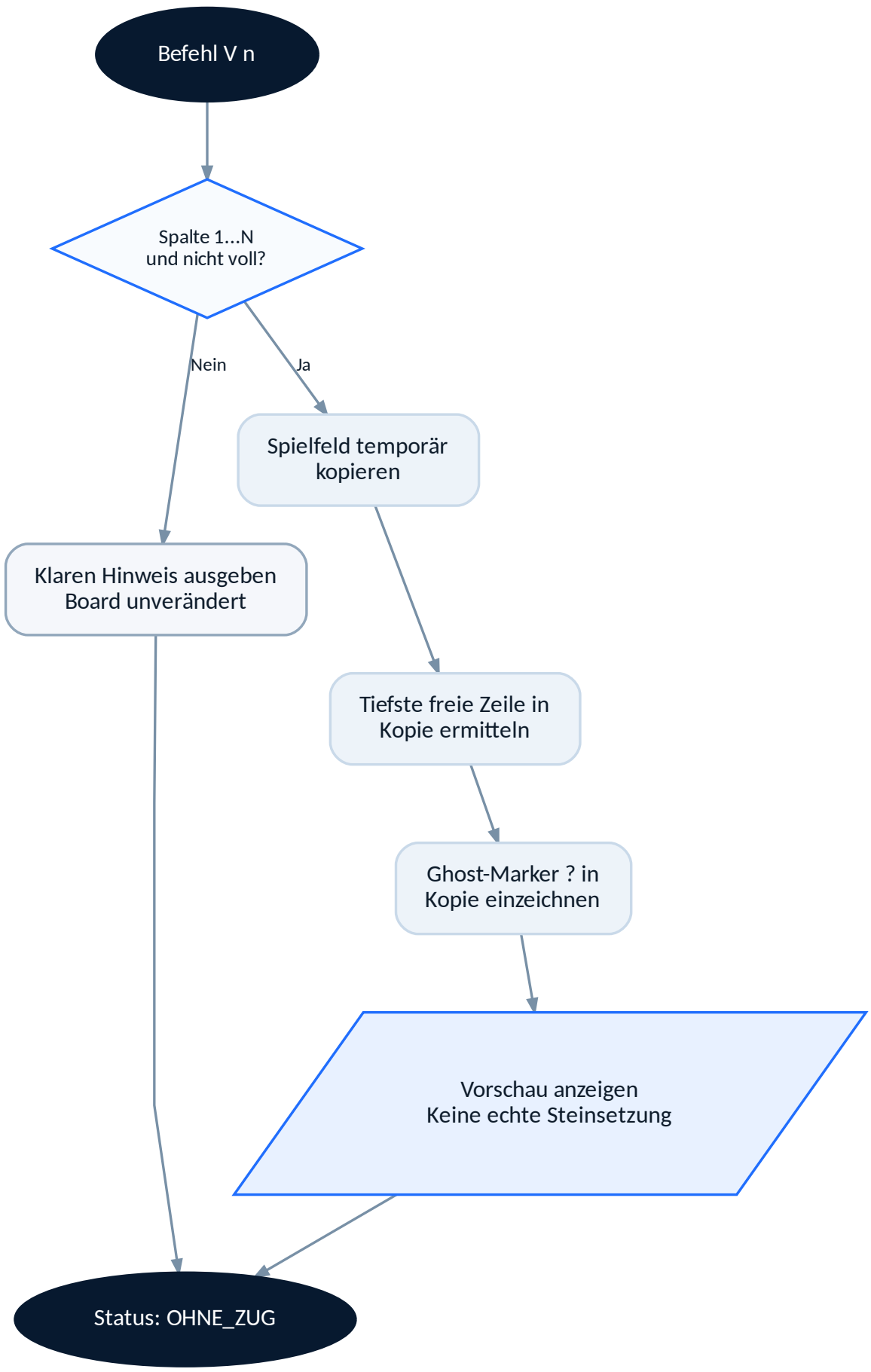
P10 / OPTIONALES FEATURE

Zugvorschau

Zeigt einen temporären Marker an der Landeposition.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / LESEN & ERKLÄREN

ZIEL

Eine Vorschau ist eine Sichtdarstellung und kein Spielzug.

EINGABE

Befehl V und Spaltennummer

ERGEBNIS

Markiertes Vorschaubrett

SICHERHEIT

Originalbrett bleibt vollständig unverändert.

NACHWEIS

Nach Vorschau kann dieselbe Person den echten Zug setzen.

KURZABLAUF

01 Spalte prüfen

02 Board kopieren

03 Freie Landereihe ermitteln

04 Vorschau ohne Steinsetzung ausgeben

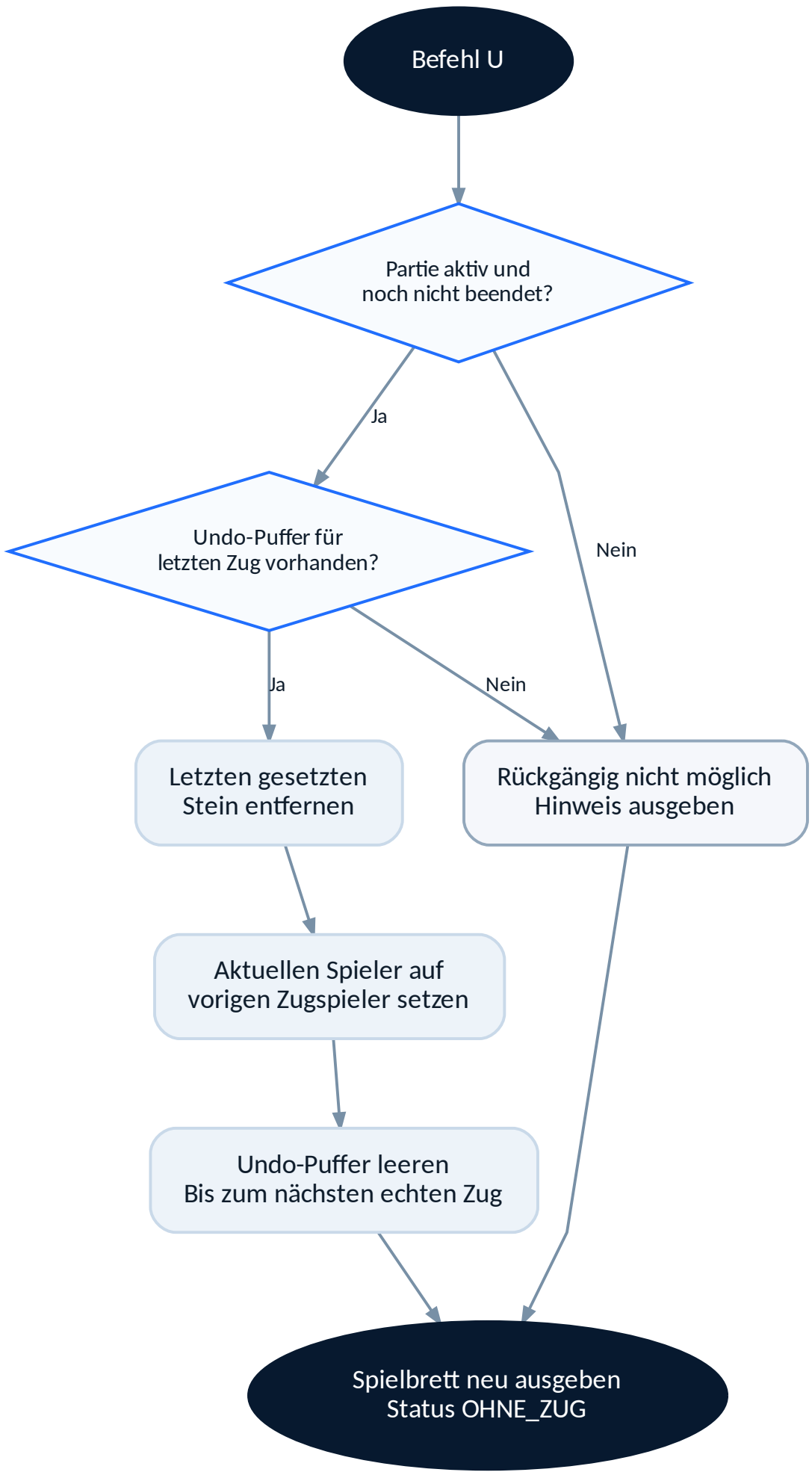
P11 / OPTIONALES FEATURE

Letzten Zug zurücknehmen

Ermöglicht genau einen Rücksprung im aktiven Spiel.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / LESEN & ERKLÄREN

ZIEL

Ein begrenzter Undo-Puffer ist klarer und leichter testbar als unbegrenzte Spielhistorie.

EINGABE

Befehl U

ERGEBNIS

Board und Spielerzustand zurückgesetzt

SICHERHEIT

Nach Spielende und nach Laden ist Undo deaktiviert.

NACHWEIS

Zweites U ohne neuen gültigen Zug wird abgewiesen.

KURZABLAUF

01 Spielstatus prüfen

02 Letzten gültigen Zug ermitteln

03 Letzten Stein entfernen

04 Vorigen Spieler wieder aktivieren

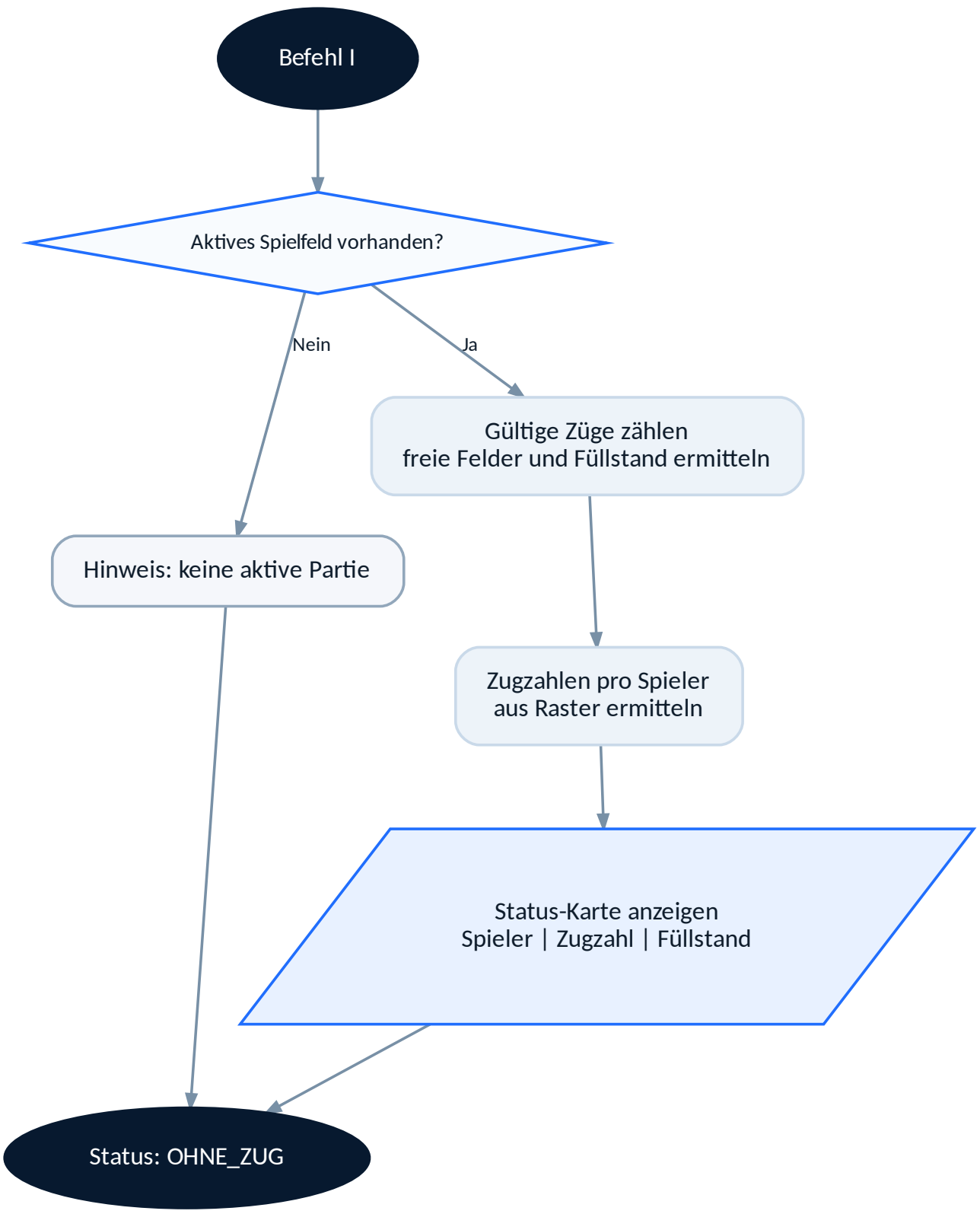
P12 / OPTIONALES FEATURE

Match Insights

Zeigt belegte Felder, freie Felder und Spielerstatistik.

01 / ABLAUFDIAGRAMM

Vektorformat · ohne feste Auflösung



02 / TECHNISCHE LESEHILFE

ZIEL

Die Kennzahlen stammen direkt aus dem Brett statt aus fehleranfälligen Parallelzählern.

KONTROLLPUNKT

Ohne laufende Partie nur einen Hinweis ausgeben.

ABNAHMETEST

Die angezeigten Summen entsprechen exakt dem Board.